

Report progetto Architetture Dati: data drift su ImageNet

Daniele Besozzi 894998, Andrea Consonni 900116, Elena Zen 895355

Indice

1	Introduzione al problema	2
1.1	Definizioni di data drift	2
1.2	Tipologie di data drift	3
1.2.1	Covariate shift (o Population Drift)	3
1.2.2	Prior probability shift	3
1.2.3	Concept Shift (o Concept Drift)	4
1.3	Cause del drift	4
2	Analisi dei dataset	5
2.1	ImageNet	5
2.2	ImageNet-1k	5
2.3	ImageNet-C	6
2.4	ImageNet-R	7
3	Introduzione al modello	9
3.1	Reti neurali convolutive	9
3.2	Reti neurali residuali	11
3.2.1	ResNet50	12
4	Presentazione della metodologia	13
4.1	Librerie e strumenti utilizzati	13
4.2	Preprocessing dei dataset	13
4.3	Estrazioni delle feature e inferenza del modello	14
4.4	Pipeline di testing e calcolo delle metriche	14
4.5	Analisi del drift	15
4.6	Analisi delle metriche di performance	16
5	Analisi dei risultati	17
5.1	Analisi delle metriche di drift sui dataset	17
5.1.1	ImageNet-1k vs ImageNet-R	17
5.1.2	ImageNet-1k vs ImageNet-C	17
5.2	Analisi delle metriche di drift sulle feature del modello	41
5.2.1	ImageNet-1k vs ImageNet-R	41
5.2.2	ImageNet-1k vs ImageNet-C	42
5.3	Analisi delle performance del modello	46
5.3.1	ImageNet-1k vs ImageNet-R	46
5.3.2	ImageNet-1k vs ImageNet-C	46

Capitolo 1

Introduzione al problema

I sistemi di Machine Learning vengono generalmente sviluppati assumendo che i dati osservati durante l'addestramento siano rappresentativi di quelli che il modello incontrerà durante il suo utilizzo. Nelle applicazioni reali, tuttavia, questa assunzione è spesso disattesa: le caratteristiche della distribuzione dei dati possono variare nel tempo o tra differenti contesti di utilizzo. Questo fenomeno, comunemente indicato come **data drift** o **distribution shift**, può determinare una significativa riduzione delle prestazioni del modello anche in assenza di modifiche alla sua architettura o ai suoi parametri.

Lo studio della robustezza rispetto al **data drift** rappresenta pertanto un aspetto fondamentale nella valutazione di sistemi di Machine Learning. È infatti importante non limitarsi a misurare le prestazioni su dati provenienti dalla stessa distribuzione utilizzata in fase di training (**in-distribution**), ma valutare anche il comportamento del modello in presenza di variazioni nella distribuzione degli input (**out-of-distribution**). Tali variazioni possono assumere forme differenti, includendo cambiamenti nelle condizioni di acquisizione delle immagini, nella loro qualità, nello stile visivo o nella presenza di specifiche perturbazioni.

In questo progetto, l'effetto del **data drift** sulle prestazioni dei modelli di classificazione d'immagini è stato analizzato attraverso una serie di benchmark basati sulla famiglia di dataset ImageNet. In particolare, sono stati considerati **ImageNet**, utilizzato come riferimento per la valutazione **in-distribution**, **ImageNet-R**, progettato per valutare la robustezza del modello rispetto a variazioni nella rappresentazione visiva degli oggetti, e **ImageNet-C**, che introduce diverse tipologie di corruzioni comuni applicate alle immagini. La combinazione di questi benchmark consente di analizzare il comportamento del modello in presenza di forme differenti di **distribution shift**, distinguendo tra variazioni di stile e rappresentazione e perturbazioni che simulano degradazioni o condizioni avverse nell'acquisizione dei dati.

L'obiettivo dell'analisi è quindi quantificare quanto il cambiamento nella distribuzione degli input influenzi le prestazioni del modello rispetto allo scenario di riferimento.

1.1 Definizioni di data drift

Nel machine learning supervisionato applicato a scenari reali, l'assunzione fondamentale per garantire la capacità di generalizzazione di un modello è che i dati di addestramento e i dati su cui il modello opererà una volta rilasciato siano indipendenti e identicamente distribuiti (i.i.d.) [1], [2]. Tuttavia, negli scenari applicativi reali, questa assunzione teorica viene costantemente violata. Le discrepanze tra l'ambiente controllato di laboratorio e le condizioni dinamiche del mondo reale, indotte da fattori quali distorsioni nella raccolta dei dati (sample selection bias), ambienti non stazionari o evoluzioni temporali e spaziali, causano variazioni nelle distribuzioni dei dati.

La letteratura scientifica ha storicamente sofferto di una frammentazione terminologica, utilizzando in modo intercambiabile nomi diversi per definire concetti analoghi (es. concept drift, population drift, changing environments, data fracture) [1]. In questo progetto adottiamo la seguente definizioni proposte da Moreno-Torres et al. [1]:

Definizione 1 (Dataset shift). *Si verifica quando la distribuzione di probabilità congiunta delle feature (o covariate) x e della variabile target (classe) y differisce tra la fase di addestramento (P_{tr}) e la fase di test (P_{tst}). Formalmente, $P_{tr}(y, x) \neq P_{tst}(y, x)$.*

Per poter analizzare e risolvere questo problema, è necessario distinguere la direzione causale tra la classe y e le feature x . Si identificano quindi due classi di problemi:

- Problemi $X \rightarrow Y$, in cui la classe y è determinata causalmente dal valore delle feature x (es. la rilevazione delle frodi bancarie, dove il comportamento dell'utente determina se c'è una frode).
- Problemi $Y \rightarrow X$, in cui la classe y determina causalmente il valore delle feature x (tipico della diagnostica medica, in cui la patologia y causa i sintomi osservabili x)

1.2 Tipologie di data drift

Il data drift può manifestarsi in diverse forme, a seconda della natura delle variazioni nella distribuzione dei dati. In questa sezione presentiamo le principali tipologie di data drift identificate in letteratura.

1.2.1 Covariate shift (o Population Drift)

Il covariate shift si riferisce al cambiamento della distribuzione delle variabili di input (o covariate) x tra la fase di training e quella di test, mentre la relazione condizionata che mappa l'input alla classe rimane del tutto invariata. Formalmente:

Definizione 2 (Covariate shift). *Appare solamente nei problemi del tipo $X \rightarrow Y$ e si verifica nel caso in cui $P_{tr}(x) \neq P_{tst}(x)$ e $P_{tr}(y|x) = P_{tst}(y|x)$*

Ad esempio, un modello di classificazione d'immagini addestrato principalmente su foto di cani su uno sfondo erboso che viene poi testato su immagini di cani che indossano impermeabili in contesti urbani [2]. Le feature visive di sfondo e i dettagli superficiali cambiano ($P(x)$ muta), ma il concetto semantico di "cane" associato a quelle immagini rimane lo stesso ($P(y|x)$ è costante).



Figura 1.1: Esempio di covariate shift

1.2.2 Prior probability shift

Il prior probability shift rappresenta il caso duale del covariate shift, focalizzato sul cambiamento della distribuzione della variabile target y , mentre la distribuzione condizionata delle feature data la classe rimane costante. Formalmente:

Definizione 3 (Prior probability shift). *Appare solamente nei problemi del tipo $Y \rightarrow X$ e si verifica nel caso in cui $P_{tr}(y) \neq P_{tst}(y)$ e $P_{tr}(x|y) = P_{tst}(x|y)$.*

Ad esempio, in un sistema di diagnostica medica per una specifica malattia infettiva, la distribuzione dei sintomi per i soggetti malati e sani rimane costante ($P(x|y)$ non cambia), ma la prevalenza complessiva della malattia nella popolazione $P(y)$ fluttua nel tempo (ad esempio, a causa di un'improvvisa epidemia o di una campagna vaccinale).

1.2.3 Concept Shift (o Concept Drift)

Il concept shift (o concept drift) si verifica quando cambia la relazione funzionale o semantica tra le feature di input e la variabile target, modificando di fatto la definizione stessa della classe da apprendere. Rappresenta la sfida più complessa, poiché i criteri decisionali appresi dal modello durante l'addestramento non sono più validi nell'ambiente operativo. Formalmente:

Definizione 4 (Concept shift). *Si verifica nel caso in cui:*

$$\begin{aligned} P_{tr}(y|x) \neq P_{tst}(y|x) \quad e \quad P_{tr}(x) = P_{tst}(x) \quad & (\text{nei problemi } X \rightarrow Y) \\ P_{tr}(x|y) \neq P_{tst}(x|y) \quad e \quad P_{tr}(y) = P_{tst}(y) \quad & (\text{nei problemi } Y \rightarrow X) \end{aligned}$$

Ad esempio, nei sistemi di filtraggio dello spam o di rilevamento delle intrusioni di rete, gli utenti malintenzionati modificano attivamente le proprie strategie per eludere i filtri esistenti, cambiando la definizione di ciò che costituisce un'e-mail di spam o un attacco, lasciando però apparentemente inalterata la struttura generale del traffico dati.

1.3 Cause del drift

Si possono rilevare due principali cause del data drift:

- **Sample Selection Bias:** Avviene quando il metodo di campionamento o di etichettatura dei dati di addestramento non è uniforme o casuale rispetto alla popolazione reale.
- **Ambienti Non Stazionari :** Molti contesti operativi sono intrinsecamente dinamici nel tempo e nello spazio. In generale, gli ambienti sono dinamici e, talvolta, la difficoltà nel far corrispondere lo scenario di apprendimento all'utilizzo nel mondo reale è dovuta proprio a questi cambiamenti. Tali scenari rendono complesso valutare l'adeguatezza di uno specifico modello in presenza di variazioni ambientali, aumentando così la probabilità di shift.

Capitolo 2

Analisi dei dataset

Per valutare e quantificare la capacità di generalizzazione Out-of-Distribution (OOD) del modello analizzato in questo progetto, abbiamo strutturato i test utilizzando tre dataset della famiglia ImageNet. In particolare, abbiamo utilizzato i dataset ImageNet-1k, ImageNet-C e ImageNet-R. In questo capitolo, descriveremo le caratteristiche principali di ciascun dataset e le motivazioni che ci hanno portato a sceglierli per i nostri test.

2.1 ImageNet

ImageNet è un ampio dataset d'immagini etichettate, ampiamente utilizzato per addestrare e valutare modelli di visione artificiale. La versione originale di Imagenet viene chiamata ImageNet-21K (o ImageNet-22K) e contiene circa 14,2 milioni d'immagini distribuite su 21.841 classi gerarchiche. La costruzione del dataset è stata effettuata in tre fasi:

1. **Mappatura WordNet:** per strutturare il dataset è stata usata la struttura gerarchica di WordNet, un database lessicale che organizza le parole in insiemi di sinonimi (synsets) e le collega tra loro attraverso relazioni semantiche. Ogni synset ha un "WordNet ID" (WNID) univoco, che è stato utilizzato per etichettare le immagini nel dataset. In questo modo, le immagini sono state organizzate in categorie gerarchiche basate sui synset di WordNet. La tassonomia si sviluppa su 9 livelli gerarchici, partendo da concetti astratti di alto livello (es. level 1: mammal) fino a classi estremamente specifiche (es. level 9: German shepherd)
2. **Web scraping:** le immagini sono state raccolte da Internet utilizzando tecniche di web scraping. Sono stati utilizzati motori di ricerca e altre fonti online per raccogliere immagini pertinenti alle categorie di oggetti definite dai synset di WordNet.
3. **Annotazione manuale:** le immagini raccolte sono state annotate manualmente attraverso il servizio Amazon Mechanical Turk (AMTurk). Ogni immagine è stata etichettata con la categoria di oggetti corretta, garantendo così l'accuratezza delle annotazioni. 49.000 lavoratori provenienti da 167 paesi hanno validato e filtrato oltre 160 milioni d'immagini candidate.

Non è presente uno split training-validation-test ufficiale. Il dataset non è bilanciato: alcune classi contengono un numero molto basso di esempi, mentre altre classi contengono un numero elevato di esempi.

2.2 ImageNet-1k

Vengono spesso utilizzati subset di ImageNet-21k per vari scopi. Uno dei più noti è ImageNet-1k, che contiene 1.000 classi di oggetti selezionate da ImageNet-21k. Il dataset contiene 1.281.167 immagini di training, 50.000 immagini (50 per classe) di validation e 100.000 immagini di test.

Questo dataset è stato utilizzato per addestrare e valutare modelli di visione artificiale, in particolare per la competizione ImageNet Large Scale Visual Recognition Challenge (ILSVRC). Per questo motivo, il test set non è etichettato, in quanto serve unicamente a valutare le prestazioni dei modelli durante la competizione. Perciò, per i nostri test, abbiamo utilizzato il validation set come test set, questo è

possibile in quanto il validation set non viene utilizzato per l'addestramento del modello, ma solo per la valutazione delle sue prestazioni[3].

A differenza della struttura gerarchica ad albero di ImageNet-21K (che include macro-classi come "mammifero" o "veicolo"), ImageNet-1K contiene esclusivamente 1.000 classi non sovrapposte poste alla base dell'albero gerarchico (es. singole razze di cani come "German shepherd", senza includere la classe genitore "dog")

2.3 ImageNet-C

A differenza di altri dataset, ImageNet-C non è stato raccolto fotografando nuovi oggetti o facendo web scraping. È stato costruito applicando sistematicamente corruzioni digitali e matematiche direttamente sul set di validazione originale di ImageNet-1K (le 50.000 immagini di validazione). L'obiettivo dei creatori era imitare le reali limitazioni fisiche dei sensori delle fotocamere, le variazioni atmosferiche ambientali e i disturbi dovuti alla compressione e trasmissione dei dati, escludendo però alterazioni avversariali mirate (worst-case)[4]. Il dataset si sviluppa in modo gerarchico:

- **Corruzioni:** sono state applicate 15 corruzioni digitali e matematiche, suddivise in 4 categorie principali: *blur*, *noise*, *digital* e *weather*.
 - *Blur*:
 - * Defocus Blur: Si verifica quando l'obiettivo della fotocamera è fuori fuoco rispetto al soggetto.
 - * Frosted Glass Blur: Riproduce la diffrazione della luce che attraversa pannelli o finestre di vetro satinato.
 - * Motion Blur: Sfocatura da movimento dovuta allo spostamento rapido della fotocamera durante il tempo di esposizione.
 - * Zoom Blur: Sfocatura radiale causata dal rapido avvicinamento fisico (o variazione di focale) verso l'oggetto.
 - *Noise*:
 - * Gaussian Noise: Simula il rumore termico e di bassa luminosità tipico dei sensori fisici.
 - * Shot Noise (Poisson Noise): Rumore elettronico causato dalla natura discreta della luce stessa durante l'acquisizione dei fotoni.
 - * Impulse Noise: Equivalente a colori del rumore "sale e pepe", causato da errori di trasmissione dei bit nei canali digitali.
 - *Digital*:
 - * Contrast: Modifica l'intervallo dinamico dei colori, simulando condizioni di luce estrema o scarsa riflettanza dei materiali.
 - * Elastic Transformations: Deforma localmente l'immagine stirando o contraendo piccole aree di pixel, simulando distorsioni ottiche della lente.
 - * Pixelation: Riproduce la perdita di dettagli ad alta frequenza che si verifica quando un'immagine a bassissima risoluzione viene ingrandita.
 - * JPEG: Introduce artefatti a blocchi e rumore ad alta frequenza tipici della compressione d'immagine lossy.
 - *Weather*:
 - * Snow: Simula la precipitazione nevosa, che introduce occlusioni e rifrazioni di luce complessa.
 - * Frost: Riproduce la formazione di cristalli di ghiaccio sulla superficie della lente o del sensore.
 - * Fog: Avvolge gli oggetti simulando la nebbia tramite l'algoritmo diamond-square, che assicura che ogni banco di nebbia generato sia unico per forma e densità su ciascuna immagine.
 - * Brightness: Simula le variazioni di intensità della luce solare diurna.
- **Intensità:** per ogni corruzione sono stati generati 5 livelli di intensità, da 1 (minima) a 5 (massima).

La dimensione totale del test set di ImageNet-C è quindi di $50.000 \times 15 \text{ corruzioni} \times 5 \text{ severità} = 3.750.000$ immagini di test.

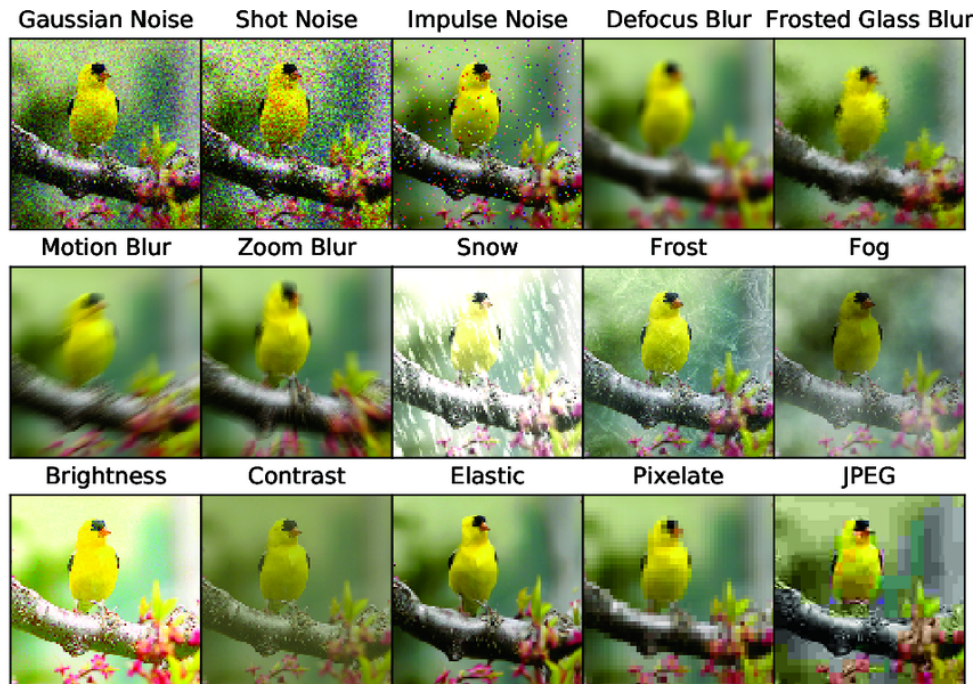


Figura 2.1: Esempi di corruzioni applicate a un'immagine del dataset ImageNet-1K.

2.4 ImageNet-R

ImageNet-R è un dataset di immagini raccolte da Internet, progettato per testare la capacità di generalizzazione dei modelli di visione artificiale su immagini stilizzate e artistiche. Il dataset è stato creato seguendo le seguenti procedure descritte in [5]:

1. **Sottocampionamento delle classi:** Gli autori hanno selezionato un subset di 200 classi di ImageNet-1K. Ciò è stato necessario a causa della difficoltà di trovare immagini stilizzate per tutte le 1.000 classi di ImageNet-1K, dunque sono state selezionate solo le classi per le quali era possibile reperire un numero sufficiente d'immagini stilizzate. Inoltre sono state scelte classi facilmente riconoscibili da un umano, ad esempio "cane" invece che indicare una razza specifica di cane.
2. **Raccolta delle immagini:** Le immagini sono state ottenute tramite scraping da Flickr, fonte da cui erano state ottenute parte delle immagini di ImageNet-1K.
3. **Etichettatura:** Le immagini sono state etichettate associando il nome dell'oggetto a termini relativi a stili e supporti artistici, come: "art", "cartoon", "graffiti", "embroidery", "graphics", "origami", "painting", "pattern", "plastic object", "plush object", "sculpture", "line drawing", "tattoo" e "toy".
4. **Filtro di qualità manuale:**
 - Gli annotatori di Amazon Mechanical Turk hanno filtrato i dati grezzi tramite un'interfaccia modificata ereditata dal progetto ImageNetV2. Ad esempio, dopo lo scraping della query "lighthouse cartoon", i lavoratori dovevano validare se l'immagine rappresentasse effettivamente una rendition corretta e non ambigua di un faro.
 - Per eliminare ogni residuo di rumore, studenti di dottorato hanno esaminato manualmente ogni singola immagine selezionata, escludendo quelle con etichette errate o che contenevano soggetti multipli appartenenti a classi diverse di ImageNet.

Il dataset finale contiene 30.000 immagini di test, con un numero variabile d'immagini per classe, da un minimo di 51 a un massimo di 430 immagini per classe, in media 150 immagini per classe. Le immagini sono categorizzate in base a diversi stili (es. Painting, Sculpture, Origami, Cartoon, Toy, Embroidery,

ecc.). Ai fini della classificazione, le etichette di destinazione rimangono quelle semantiche di ImageNet-1K.

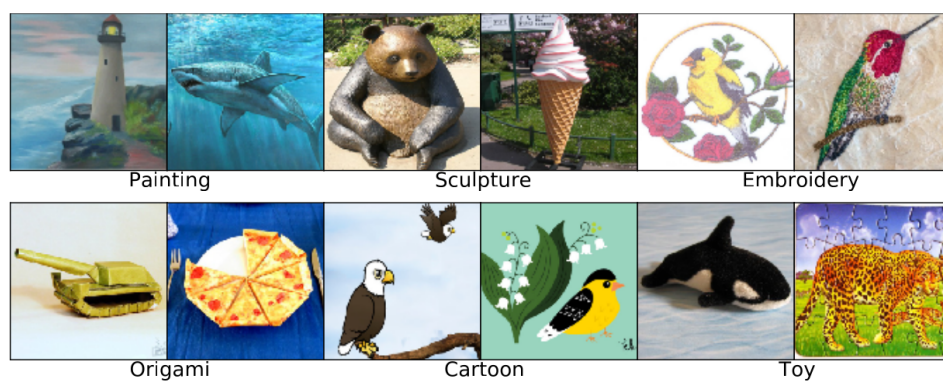


Figura 2.2: Esempi di immagini stilizzate del dataset ImageNet-R. Painting, etc. sono stili di rappresentazione, non etichette.

Capitolo 3

Introduzione al modello

Ogni tipo di modello di machine learning supervisionato può riscontrare, quando si trova ad operare in contesti reali, un drift nelle distribuzioni dei dati osservati rispetto alla distribuzione osservata durante la fase di addestramento. In particolare, uno dei tipi di modello più diffuso per il riconoscimento delle immagini è quello che prende il nome di **rete neurale convolutiva** (Convolutional neural network, CNN). In questo capitolo, introdurremo il funzionamento delle CNN e descriveremo l'architettura utilizzata per questo progetto, le **reti neurali residueali**.

3.1 Reti neurali convolutive

Le **reti neurali convolutive** furono per la prima volta introdotte nella loro forma moderna nel 1989 da Le Cun et al [6], i quali svilupparono LeNet-5, una rete convolutiva in grado di classificare le cifre scritte a mano su dei fogli. La struttura particolare di questo tipo di reti le rende particolarmente efficaci nell'ambito del riconoscimento di immagini, poiché esse sono capaci di sfruttare le **caratteristiche spaziali dell'input**, come per esempio la correlazione spaziale tra pixel vicini. Questa caratteristica permette inoltre di **ridurre il numero di pesi necessari da imparare**, rendendo quindi la rete più efficiente. La figura 3.1 mostra l'architettura di LeNet-5 come esempio di rete neurale convolutiva. Oltre ai layer di input e di output, le reti neurali convolutive sono formate principalmente da tre tipi di strati [7]:

- Strati convolutivi
- Strati di pooling (o di subsampling)
- Strati completamente connessi

Strato di input

Come nelle reti neurali tradizionali, lo strato di input di una rete neurale convolutiva conterrà i valori delle istanze che verranno passate alla rete. In particolare per le immagini, essi sono i **valori numerici**

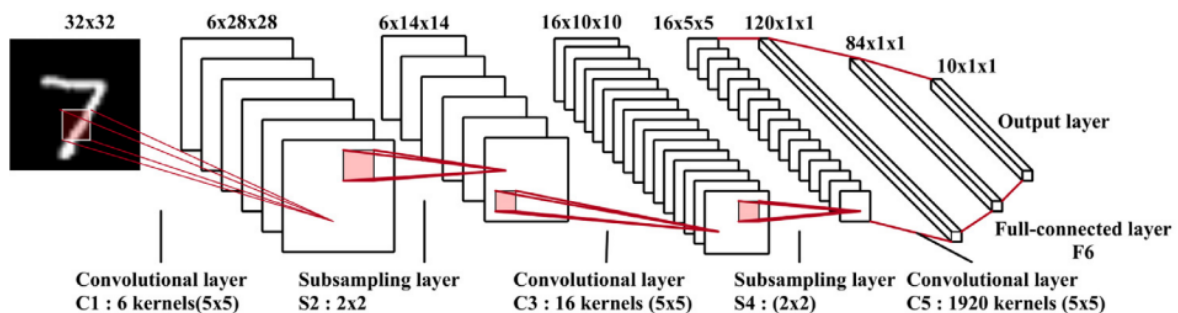


Figura 3.1: Architettura di LeNet-5. Da [8]

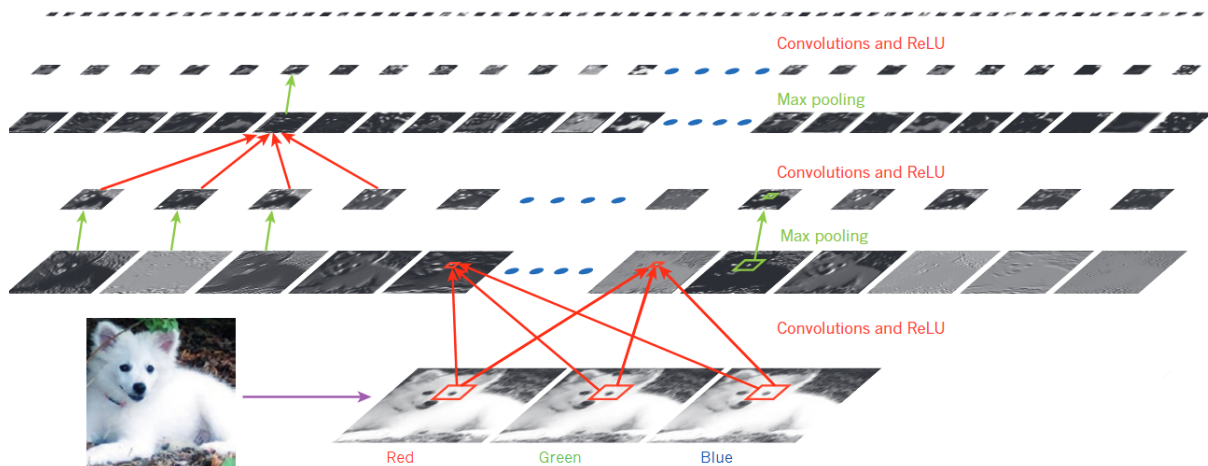


Figura 3.2: Esempio di estrazione di feature map da un'immagine. Da [9]

dei suoi pixel. Possiamo quindi vedere un'immagine come una matrice di dimensioni $W \times H \times C$, con W la larghezza dell'immagine, H l'altezza dell'immagine e C il numero di **canali colore** (per esempio, $C = 3$ se l'immagine ha canali colore RGB).

Strati convolutivi

Come abbiamo già detto sopra, le reti neurali convolutive sono in grado di sfruttare la struttura spaziale dell'input. Questo avviene nei **layer convolutivi**, il cui scopo è quello di imparare delle **rappresentazioni delle feature dell'input** [8], cioè una rappresentazione delle caratteristiche presenti all'interno dell'input. Nel contesto delle immagini, esse possono essere, ad esempio, bordi, angoli, texture e forme. L'apprendimento e l'estrazione di tali caratteristiche vengono effettuati avvalendosi di diversi **kernel convolutivi** (o filtri convolutivi), i quali vengono appresi dalla rete e utilizzati per effettuare una **convoluzione** sull'immagine. Più precisamente, il kernel viene fatto scorrere sull'input, considerando a ogni posizione una piccola regione locale di esso. Tale regione, rispetto al neurone che ne riceve le informazioni, viene definita **receptive field**. Viene quindi effettuato il prodotto scalare tra i valori presenti in questa regione e i pesi del kernel, ottenendo il valore di ingresso del neurone, il quale applicherà ad esso una funzione di attivazione non lineare per ottenere il suo valore di output, cioè la sua **attivazione**. Ripetendo questa operazione nelle diverse posizioni spaziali dell'input si ottiene una **feature map**, nella quale ogni valore corrisponde all'attivazione di un neurone in corrispondenza di una specifica regione dell'input. Ogni layer convolutivo si avvale di più kernel, i quali produrranno una feature map diversa ciascuno. Questo meccanismo ha diversi vantaggi:

- Lo stesso kernel viene **condiviso tra tutte le posizioni spaziali dell'input**: in questo modo, lo stesso insieme di pesi viene utilizzato per rilevare una determinata caratteristica indipendentemente dalla sua posizione nell'immagine. Ciò inoltre diminuisce la complessità del modello, facilitandone l'addestramento.
- L'utilizzo di kernel differenti permette di ottenere diverse feature map, ciascuna delle quali può rappresentare una diversa caratteristica appresa dall'input

La figura 3.2 mostra un esempio di estrazione di feature maps in una CNN. Infine, il kernel viene spostato sull'immagine di una quantità chiamata **stride**, che rappresenta il numero di pixel di cui il kernel viene traslato a ogni operazione di convoluzione. Di conseguenza, aumentando lo stride diminuisce il numero di posizioni nelle quali il kernel viene applicato e, in generale, si ottiene una **feature map** di dimensioni spaziali inferiori. Se i kernel della rete sono troppo grandi rispetto alle dimensioni dell'input, si può inoltre introdurre dello **zero padding** ai suoi bordi.

Strati di pooling

Lo scopo degli strati di pooling è quello di **ridurre le dimensioni spaziali delle feature map ottenute dai layer convolutivi** e, di conseguenza, ridurre la complessità computazionale del modello [7]. Anche

questi layer si avvalgono di kernel per operare su piccole regioni spaziali dell'input. Tuttavia, a differenza degli strati convolutivi, questi kernel non possiedono parametri apprendibili e applicano generalmente una delle seguenti operazioni:

- **Max-pooling:** viene selezionato il valore massimo tra quelli contenuti nella regione considerata.
- **Average-pooling:** viene calcolata la media dei valori contenuti nella regione considerata.

L'output di un layer di pooling sarà quindi una nuova **feature map** con dimensioni spaziali ridotte rispetto a quella in ingresso. I layer di pooling sono generalmente posti tra due layer convolutivi e operano indipendentemente su ciascuna feature map. Combinando diversi layer convolutivi e di pooling, la rete può progressivamente estrarre caratteristiche sempre più complesse e di alto livello [8].

Strati completamente connessi

Come in una rete neurale tradizionale, gli strati completamente connessi si occupano di **combinare le informazioni provenienti dallo strato precedente**, collegando ciascun neurone a tutti i neuroni dello strato precedente. In questo modo, le caratteristiche estratte dai layer convolutivi e vengono integrate per ottenere una rappresentazione globale dell'input [8], che può essere successivamente utilizzata per effettuare la classificazione.

Strato di output

L'ultimo strato di una rete neurale convolutiva è lo strato di output, il quale si occupa di restituire il risultato della classificazione. Spesso, questo strato è rappresentato dall'operatore **softmax** oppure da una **SVM** [8].

3.2 Reti neurali residuali

Le **reti neurali residuali** sono un'architettura per reti neurali profonde, proposto per la prima volta nel 2016 da He et al. [3] e progettata per affrontare il problema della **degradazione**. Tale fenomeno si manifesta quando l'aumento della profondità della rete, ovvero del numero di layer, porta inizialmente a una saturazione dell'accuratezza e successivamente a un suo rapido decremento. Questo problema, inoltre, non è causato dall'overfitting, quindi aggiungere più layer alla rete porta ad un errore maggiore durante l'addestramento. Per combattere questo fenomeno, le reti residuali adottano le seguenti soluzioni:

- **Apprendimento residuale:** Consideriamo un insieme di layer di una rete neurale e supponiamo di voler fare in modo che essi apprendano un determinato mapping $\mathcal{H}(\mathbf{x})$, dove $\mathbf{x}$ rappresenta l'input fornito al primo dei layer considerati. Anziché apprendere direttamente il mapping desiderato, facciamo apprendere ai layer il **mapping residuale** $\mathcal{F}(\mathbf{x})$, definito come:

$$\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$$

Di conseguenza, il mapping originale diventa:

$$\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}) + \mathbf{x}$$

Questa riformulazione è giustificata dalla natura controintuitiva del problema della degradazione: la manifestazione di questo problema potrebbe indicare che i solver di ottimizzazione faticano ad approssimare l'identità componendo più layer non lineari. Riformulando il mapping in questo modo, l'ottimizzazione diventa più semplice: se il mapping ottimale è vicino all'identità, è sufficiente che i pesi dei layer tendano a zero.

- **Identity shortcut connections:** Il mapping $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ può essere realizzato da una rete dotata di "shortcut connections", ovvero connessioni che saltano uno o più layer della rete. Nel caso delle reti residuali, queste connessioni collegano semplicemente l'input di un gruppo di layer alla sua uscita, sommando l'identità $\mathbf{x}$ all'output dei layer.

La figura 3.3 mostra un esempio di blocco residuale. Queste soluzioni portano ad ottenere reti più facili da ottimizzare rispetto alle reti neurali profonde tradizionali e in grado di trarre maggiore beneficio, in termini di accuratezza, dall'aumento della profondità.

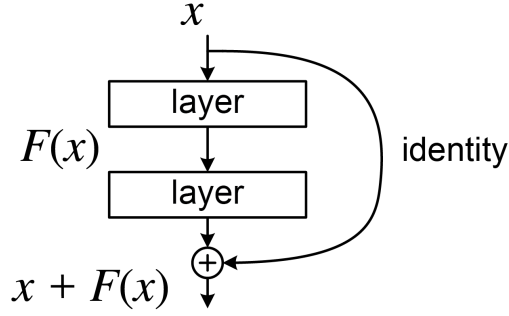


Figura 3.3: Esempio di blocco residuale. Da [3]

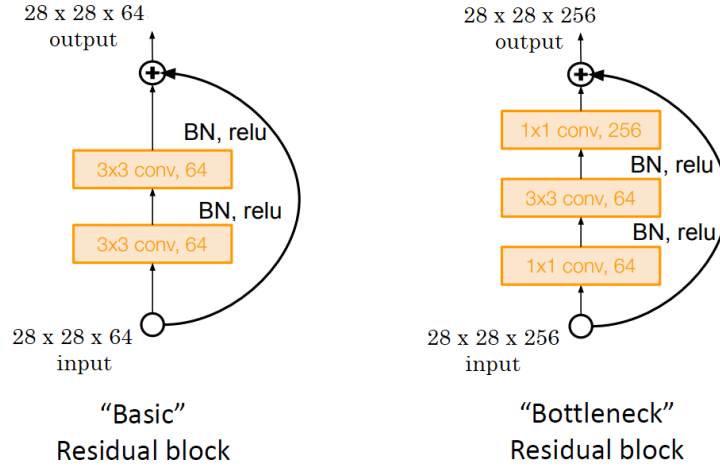


Figura 3.4: Confronto tra un blocco residuale e un blocco "bottleneck". Da [3]

3.2.1 ResNet50

I modelli sviluppati da He et al. in [3] sono stati addestrati e valutati su ImageNet-1k. Uno di questi modelli è ResNet50, il quale viene utilizzato come modello per questo progetto. Le versioni di rete residuale più profonde, tra cui ResNet50, adottano un blocco residuo differente rispetto a quello base a due layer utilizzato nelle versioni meno profonde: per contenere i tempi di addestramento, ogni funzione residuale $\mathcal{F}$ viene realizzata tramite tre layer convolutivi di dimensioni rispettivamente 1×1 , 3×3 e 1×1 , detti **layer di bottleneck**. Il primo e l'ultimo layer si occupano rispettivamente di ridurre e successivamente ricostruire le dimensioni dell'input, mentre il layer 3×3 centrale opera come un "collo di bottiglia" (*bottleneck*), con dimensioni di input e output ridotte rispetto al resto del blocco. Questo cambiamento non comporta nessun aumento significativo della complessità computazionale, la quale resta simile rispetto all'implementazione a due blocchi. Un esempio di blocco "bottleneck" è mostrato in figura 3.4

Capitolo 4

Presentazione della metodologia

In questa sezione descriviamo la metodologia adottata per valutare la capacità di generalizzazione Out-of-Distribution (OOD) del modello analizzato. Il processo si articola in tre fasi principali: preprocessing dei dataset, testing del modello e analisi del drift.

4.1 Librerie e strumenti utilizzati

Il progetto è stato sviluppato in Python 3.12 e fa uso delle seguenti librerie principali:

- **PyTorch (torch) e TorchVision:** per il caricamento del modello ResNet-50 pre-addestrato, la definizione della pipeline di preprocessing e l'esecuzione dell'inferenza sulle GPU.
- **HuggingFace Datasets:** per il caricamento, la trasformazione e la gestione efficiente dei dataset ImageNet-1k, ImageNet-R e ImageNet-C in formato Arrow, con supporto al parallelismo e alla serializzazione su disco.
- **NumPy e Pandas:** per la manipolazione numerica e tabulare dei dati, inclusa la gestione delle feature estratte e dei risultati delle metriche.
- **Scikit-learn:** per il calcolo delle metriche di classificazione (accuracy, balanced accuracy, precision, recall, F1-score, MCC, Cohen's Kappa).
- **SciPy:** per l'integrazione numerica (`trapezoid`), la regressione lineare (`linregress`) e la distribuzione t di Student per gli intervalli di confidenza.
- **Evidently:** libreria open-source per il monitoraggio e la rilevazione del data drift. È stata utilizzata per eseguire i test di Wasserstein (normed) sulle proprietà visive delle immagini e il test `empirical_mmd` sulle feature estratte dal modello.
- **Matplotlib e Seaborn:** per la visualizzazione dei risultati, inclusi grafici KDE di distribuzione delle proprietà delle immagini, andamenti delle metriche al variare della severità delle corruzioni e diagrammi a barre per il confronto baseline vs drift.
- **Pillow (PIL):** per l'estrazione delle proprietà visive delle immagini (luminosità, contrasto RMS, nitidezza, media dei canali RGB).
- **tqdm:** per le barre di progresso durante l'estrazione delle proprietà e l'esecuzione dell'inferenza.

4.2 Preprocessing dei dataset

Il preprocessing è gestito dallo script `data_preprocess/datasets_preprocess.py`, che orchestra il caricamento, la trasformazione e il salvataggio su disco dei tre dataset.

- **ImageNet-1k:** Il validation set (50.000 immagini) è stato scaricato tramite HuggingFace Datasets utilizzando il builder `parquet`, necessario poiché il builder ufficiale costringe il download dell'intero dataset mentre per il nostro progetto è sufficiente il subset di validazione. L'accesso al dataset è gated e richiede autenticazione via `huggingface_hub`.

- **Confronto con ImageNet-R:** Il validation set è stato filtrato per contenere esclusivamente le 200 classi presenti in ImageNet-R, ottenendo un dataset di 10.000 immagini (50 per classe). Questo cross-filtering garantisce che il confronto tra baseline e drift avvenga sullo stesso spazio di classi.
- **Confronto con ImageNet-C:** Il validation set è stato mantenuto intatto, in quanto ImageNet-C è costruito applicando corruzioni direttamente sulle stesse immagini del validation set di ImageNet-1k.
- **ImageNet-R:** Il dataset è stato caricato dal repository HuggingFace `axiong/imagenet-r`. Il pre-processing consiste in:
 - La mappatura dei WNID (WordNet ID) delle immagini agli indici di classe di ImageNet-1k tramite il file di mapping `LOC_sysnet_mapping.txt`, che associa ciascun WNID a un indice numerico (0-999) e a una descrizione testuale.
 - Il capping a 50 esempi per classe tramite la funzione `cap_examples_per_class()`, per garantire un bilanciamento uniforme con il set di ImageNet-1k utilizzato come baseline.
- **ImageNet-C:** Il dataset è stato scaricato direttamente dal sito Zenodo, che mette a disposizione quattro archivi tar (`blur.tar`, `digital.tar`, `noise.tar`, `weather.tar`). Dopo l'estrazione, è stata costruita una struttura di dataset HuggingFace separata per ogni coppia (corruzione, severità), ad esempio `gaussian_noise_sev_1`, `gaussian_noise_sev_2`, ecc. Ciascun dataset contiene le colonne `image`, `wnid` e `label`, dove il label è stato assegnato tramite la stessa mappatura WNID-to-index usata per gli altri dataset. I dati sono stati salvati su disco in formato Arrow per un accesso efficiente.

4.3 Estrazioni delle feature e inferenza del modello

Il modello analizzato è ResNet-50 con pesi pre-addestrati su ImageNet-1k (in particolare, i pesi `IMAGENET1K_V2`), caricato tramite `TorchVision`. Il modello è stato impostato in modalità `eval()` e non è stato sottoposto ad alcun fine-tuning: l'obiettivo è valutare la robustezza del modello senza un'addestramento o un fine-tuning degli iperparametri aggiuntivo. La funzione `extract_resnet50_features()` esegue un forward pass manuale attraverso i layer convoluzionali di ResNet-50, estraendo sia le feature del penultimo layer (output di `avgpool`, vettori a 2048 dimensioni) sia i logits di classificazione prodotti dal layer fully-connected finale. Questa strategia permette di raccogliere le feature intermedie per le analisi di drift basate sulle rappresentazioni del modello, senza dover eseguire due forward pass separati.

4.4 Pipeline di testing e calcolo delle metriche

Il testing è eseguito dagli script `test_model_ImageNetC.py` e `test_model_ImageNetR.py`, che implementano una pipeline strutturata in più step.

- **Inferenza:** La funzione `run_inference()` esegue il forward pass del modello sul dataloader, raccogliendo le etichette vere, le predizioni top-1, le predizioni top-5 e, opzionalmente, le feature del penultimo layer. Il tutto avviene con decoratore `@torch.no_grad()` per ottimizzare l'uso della memoria.
- **Metriche di classificazione:** La funzione `compute_metrics()` calcola un set completo di 14 metriche per ogni run di inferenza:
 - Accuracy e top-5 accuracy
 - Balanced accuracy (media del recall per classe)
 - Precision, recall e F1-score in varianti macro, weighted e micro
 - Matthews Correlation Coefficient (MCC)
 - Cohen's Kappa

MCC e Cohen's Kappa sono due metriche che correggono l'accuracy in modo da essere affidabili anche in presenza di dataset sbilanciati:

Il **MCC** (Matthews Correlation Coefficient) è il coefficiente di correlazione tra etichette vere e predette (vale tra -1 e $+1$, dove $+1$ indica predizione perfetta), mentre la **Cohen's Kappa** misura l'accordo tra due classificatori (qui modello vs etichette vere) al netto dell'accordo dovuto al caso, ovvero quanto le predizioni sono migliori di quelle che si otterrebbero indovinando a caso in base alla distribuzione delle classi. I loro valori non sono stati analizzati nel dettaglio, ma sono stati mantenuti per completezza.

Valutazione ripetuta con intervalli di confidenza

Per ottenere stime robuste con intervalli di confidenza, per ciascuna delle 5 ripetizioni, viene eseguito un forward pass completo applicando una pipeline di augmentation stocastica prima del preprocessing del modello. Le augmentation utilizzate sono `RandomResizedCrop(224, scale=(0.9, 1.0))` e `ColorJitter(brightness=0.08, contrast=0.08, saturation=0.05, hue=0.02)`. Questa strategia cattura la variabilità completa introdotta dalle augmentation stocastiche.

La funzione `compute_metrics_ci()` aggrega i risultati delle 5 ripetizioni calcolando media, deviazione standard e intervalli di confidenza al 95% per ogni metrica, utilizzando la distribuzione t di Student di Welch per la stima dell'intervallo.

Il **test t di Welch** (noto anche come t -test per varianze diseguali) è una variante del t -test di Student che non assume l'uguaglianza delle varianze dei due gruppi: i gradi di libertà vengono calcolati tramite la formula approssimata di Welch-Satterthwaite, che dipende sia dalle dimensioni campionarie sia dalle varianze stimate. Questo test è stato scelto perché le metriche calcolate sulle poche ripetizioni (5 run) possono presentare varianze diverse tra dataset, e questa versione restituisce un intervallo di confidenza per la differenza delle medie più robusto rispetto al t -test classico.

Baseline

Per ImageNet-C, il validation set originale di ImageNet-1k (50.000 immagini, tutte le 1000 classi) è stato utilizzato come baseline. Per ImageNet-R, il set preprocessato di ImageNet-1k (10.000 immagini, 200 classi) è stato utilizzato come baseline.

4.5 Analisi del drift

L'analisi del drift è stata condotta su due livelli:

- Proprietà visive delle immagini.
- Feature estratte dal modello ResNet-50.

Drift sulle proprietà visive delle immagini

La funzione `extract_properties()` del modulo `drift_utils.py` estrae sei proprietà visive per ogni immagine: luminosità (media della scala di grigi normalizzata), contrasto RMS (deviazione standard della scala di grigi normalizzata), nitidezza (varianza del Laplaciano, calcolata tramite il filtro `FIND_EDGES` di PIL) e le medie normalizzate dei tre canali RGB. Per ogni proprietà, il drift viene rilevato utilizzando la **distanza di Wasserstein**, implementata dalla libreria Evidently nella sua versione *normed*. La distanza di Wasserstein quantifica il costo minimo di trasporto per trasformare la distribuzione di una proprietà nel di confronto (drift) in quella del dataset di riferimento (baseline). La versione *normed* di questa metrica normalizza il valore ottenuto utilizzando la deviazione standard del baseline. Si considera presente il drift quando il punteggio supera la soglia predefinita (di default 0.1). I risultati vengono visualizzati con sovrapposizione delle distribuzioni e annotazione del punteggio e del flag di drift. Il sistema implementa un meccanismo di cache su disco (file CSV) per le proprietà estratte, evitando di ricalcolare estrazioni costose su dataset grandi. Il nome del file di cache è derivato dal nome del dataset. Per ImageNet-R, il confronto è stato effettuato utilizzando come baseline ImageNet-1k preprocessato, mentre per ImageNet-C è stata utilizzata come baseline l'intero dataset ImageNet-1k e il confronto è stato effettuato per ogni livello di severità.

Drift sulle feature del modello

Per quantificare lo spostamento distribuzionale nello spazio delle rappresentazioni estratte da ResNet-50 (feature del penultimo layer, vettori a 2048 dimensioni), è stato utilizzato il test `empirical_mmd` di Evidently. La Maximum Mean Discrepancy (MMD) empirica misura la differenza tra le medie delle

due distribuzioni nello spazio di un kernel (in genere RBF): un valore prossimo a zero indica che le due distribuzioni di feature sono praticamente identiche, mentre un valore maggiore indica la presenza di drift nelle rappresentazioni del modello. Anche in questo caso la presenza di drift viene stabilita confrontando il punteggio con una soglia predefinita.

4.6 Analisi delle metriche di performance

L'analisi delle metriche è condotta nel notebook `metrics_analysis.ipynb`, che implementa funzioni per il confronto sistematico delle prestazioni del modello tra baseline e dataset OOD.

- **Confronto ImageNet-1k vs ImageNet-R:** Per ogni metrica, viene calcolato il delta ($baseline_mean - drift_mean$) con intervallo di confidenza al 95%. L'intervallo viene stimato con due approcci alternativi: il test t di Welch (che usa gradi di libertà di Welch-Satterthwaite) quando è possibile calcolare correttamente la formula, oppure il *bootstrap* (5000 iterazioni) come alternativa. La significatività del delta è valutata verificando se lo zero è incluso nell'intervallo di confidenza.
- **Confronto ImageNet-1k vs ImageNet-C:** Per ogni tipo di corruzione e livello di severità, vengono calcolate le metriche con intervalli di confidenza. L'analisi include:
 - Plot delle metriche al variare della severità per ogni singola corruzione, con linea di baseline e bande di confidenza.
 - Plot delle metriche per categoria di corruzione (noise, blur, digital, weather), sovrapponendo le curve di tutte le corruzioni della stessa categoria.
 - Calcolo della degradazione normalizzata: $D_s = (Acc_{clean} - Acc_s) / Acc_{clean}$, dove Acc_{clean} è l'accuracy sul baseline e Acc_s l'accuracy a severità s .
 - Calcolo dell'AUC della robustezza: $AUC_{rob} = \frac{1}{5} \sum_{s=1}^5 Acc_s / Acc_{clean}$, che quantifica la capacità complessiva del modello di mantenere le prestazioni al variare della severità.
 - Calcolo della pendenza (*slope*) della degradazione tramite regressione lineare, che indica la velocità di deterioramento delle prestazioni all'aumentare della severità.
 - Calcolo dell' R^2 della regressione, che indica quanto la relazione tra severità e degradazione sia lineare.

Capitolo 5

Analisi dei risultati

In questo capitolo analizzeremo i risultati ottenuti dall'applicazione della metodologia proposta ai dataset considerati e ai risultati ottenuti dall'inferenza del modello. Verranno presentati i principali indicatori di performance, le metriche di valutazione del drift e le eventuali osservazioni emerse durante l'analisi dei dati.

5.1 Analisi delle metriche di drift sui dataset

Per analizzare l'avvenimento di un data drift rispetto ad imagenet-1k, ci avvarremo di un test statistico (Distanza di Wesserstein) sulle seguenti misure:

- Luminanza
- Contrasto
- Nitidezza (Sharpness)
- Media dei canali colore

5.1.1 ImageNet-1k vs ImageNet-R

Come si può evincere dal grafico in figura 5.1, la maggior parte delle feature presenta un drift rispetto alla distribuzione di riferimento (imagenet-1k) secondo la distanza di Wesserstein. In particolare, tutte le misure tranne sharpness presentano un drift significativo, con valori di distanza di Wesserstein superiori a 0.1. La misura di sharpness, invece, risulta essere più simile alla distribuzione di riferimento, con un valore di distanza di Wesserstein inferiore a 0.1. Possiamo inoltre osservare che la distribuzione di brightness presenta una distanza di Wesserstein molto maggiore dei canali colore. Questo potrebbe essere dovuto al fatto che essendo ImageNet-R formato da immagini artistiche, tra cui disegni è possibile che ci sia un maggior numero d'immagini con uno sfondo bianco, dunque una maggiore varianza nella luminosità.

5.1.2 ImageNet-1k vs ImageNet-C

Per quanto riguarda ImageNet-C, abbiamo osservato i risultati distinguendo le corruzioni per tipo di corruzione e severità (1-5).

Confronto delle distribuzioni delle feature: ImageNet-1k vs ImageNet-R
Drift quando WD ≥ 0.1

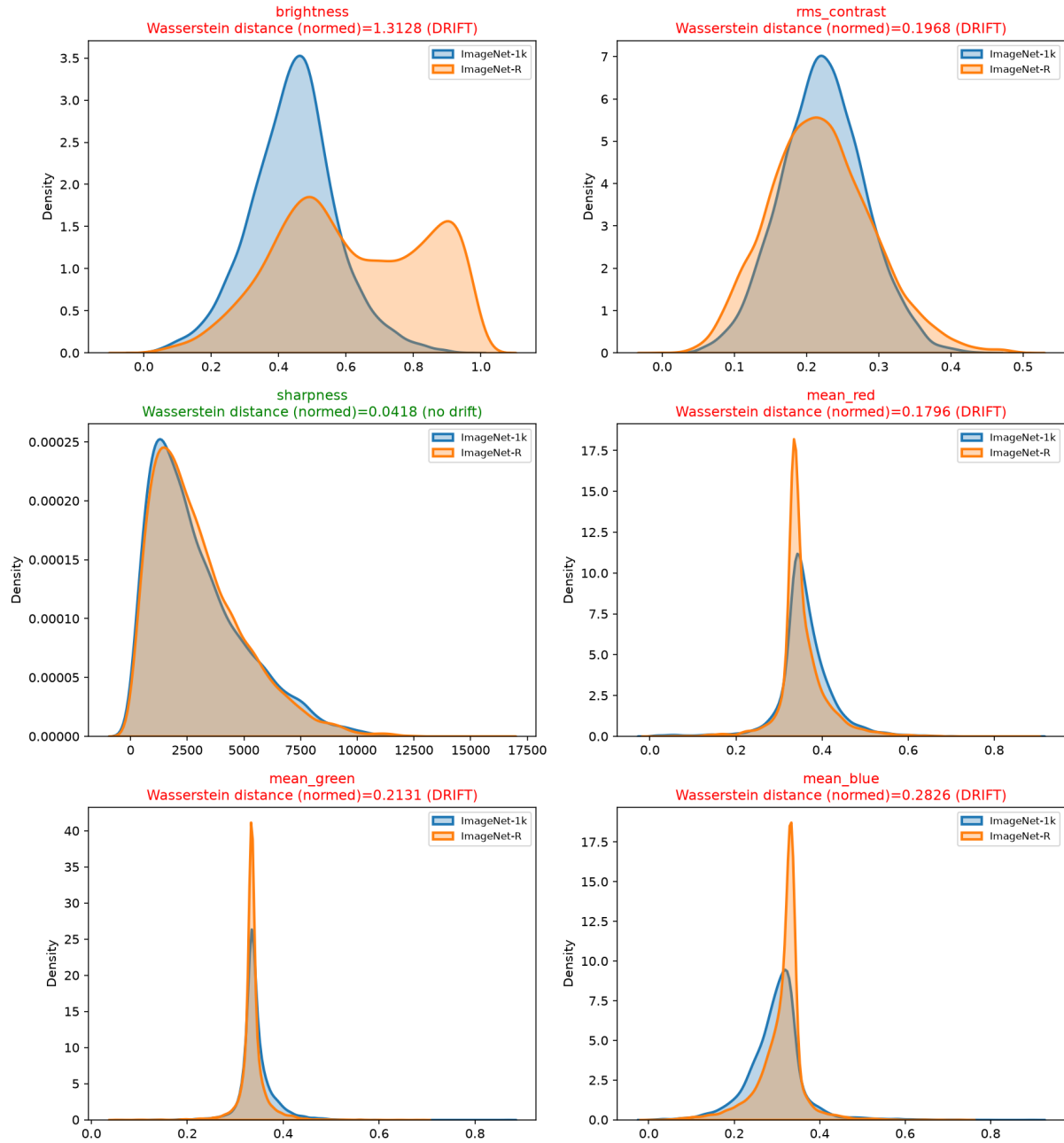


Figura 5.1: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-R.

corruption	feature	sev_1	sev_2	sev_3	sev_4	sev_5
gaussian_noise	brightness	0.072713	0.085192	0.112507	0.162998	0.251874
gaussian_noise	mean_blue	0.028428	0.040312	0.083820	0.159918	0.274003
gaussian_noise	mean_green	0.053796	0.065129	0.086815	0.128471	0.203225
gaussian_noise	mean_red	0.046630	0.043086	0.060681	0.122065	0.223818
gaussian_noise	rms_contrast	0.114575	0.126554	0.220013	0.380643	0.601419
gaussian_noise	sharpness	1.385858	2.396271	3.420685	4.200996	4.791838
impulse_noise	brightness	0.080418	0.103368	0.126473	0.189028	0.267379
impulse_noise	mean_blue	0.029314	0.061542	0.096882	0.185154	0.284710
impulse_noise	mean_green	0.058434	0.074756	0.091807	0.144506	0.209471
impulse_noise	mean_red	0.038153	0.044674	0.068579	0.144231	0.233162
impulse_noise	rms_contrast	0.116605	0.132207	0.172549	0.310412	0.488980
impulse_noise	sharpness	0.857158	1.551392	2.150133	3.356574	4.291620
shot_noise	brightness	0.073568	0.092625	0.129523	0.226492	0.337091
shot_noise	mean_blue	0.031541	0.028833	0.031633	0.075767	0.125525
shot_noise	mean_green	0.038060	0.035835	0.035784	0.056486	0.089558
shot_noise	mean_red	0.050125	0.043082	0.034105	0.055221	0.094603
shot_noise	rms_contrast	0.106031	0.139771	0.253508	0.506550	0.723193
shot_noise	sharpness	1.504179	2.558167	3.451343	4.334384	4.723952

Tabella 5.1: Distanza di Wesserstein tra le distribuzioni delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-C nel gruppo di corruzione noise.

Possiamo notare dai dati nella tabella 5.1.2 che per tutti i tipi di rumore le feature rms_contrast e sharpness driftano a tutti i livelli di severità, mentre le altre feature driftano solo a severità più alte. In particolare, la feature sharpness presenta un drift molto più marcato rispetto alle altre feature, con valori di distanza di Wesserstein che superano 4 per la severità 5. Questo, può essere casuato dal fatto che il rumore può coprire i bordi degli oggetti. Possiamo notare inoltre che mean_green e mean_red per shot_noise non driftano, mentre mean_blue drifta solo al livello di severity 5. Osservando la figura 5.2 possiamo notare che per noise la distanza nei canali colore non aumenta sempre all'aumentare della severità, questa diminuisce da severity 1 a severity 3. La distanza nelle altre feature aumenta invece sempre per tutti i rumori all'aumentare della severità.

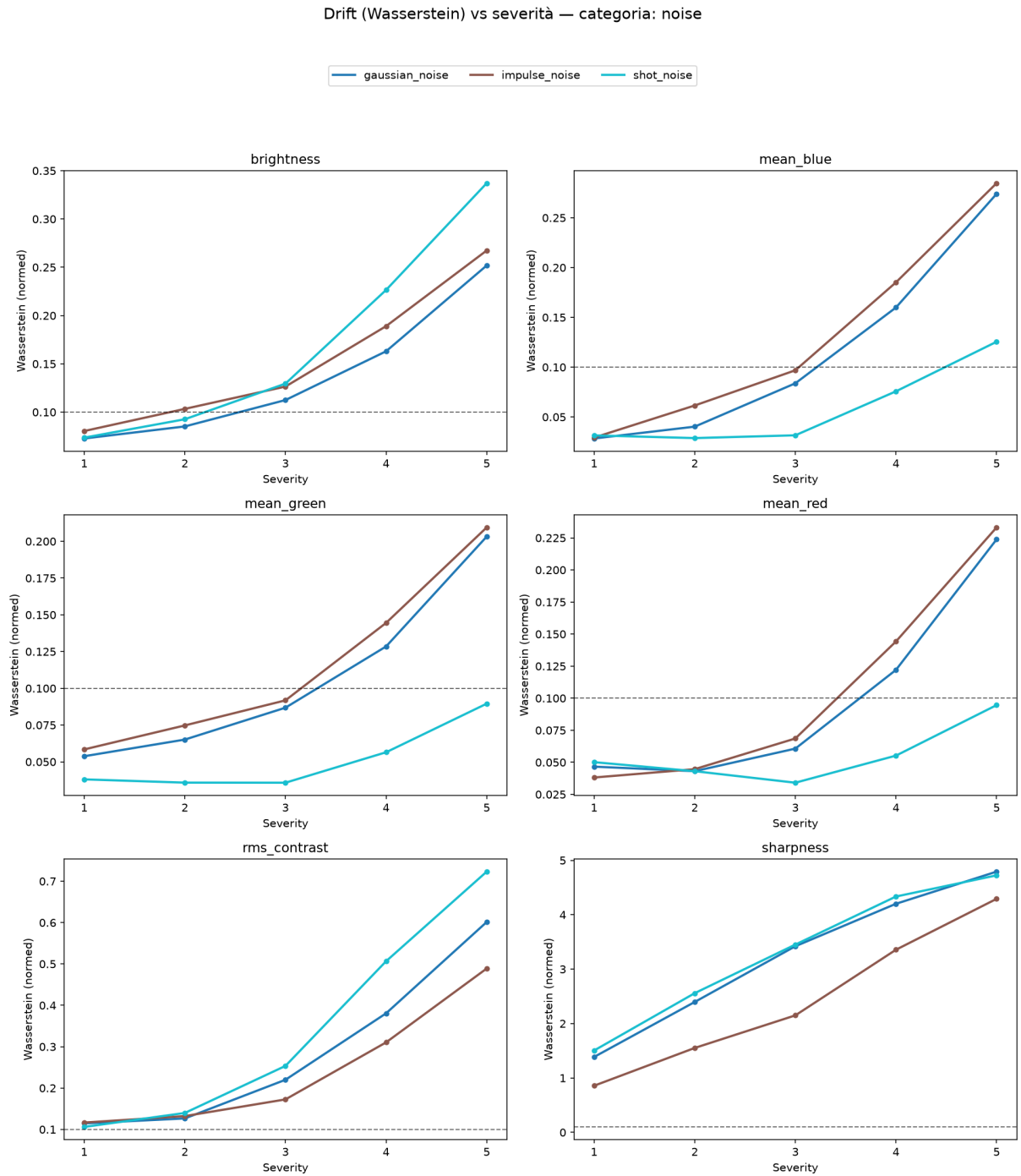


Figura 5.2: Distanza di Wesserstein tra le distribuzioni delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-C nel gruppo di corruzione noise, al variare della severit .

corruption	feature	sev_1	sev_2	sev_3	sev_4	sev_5
defocus_blur	brightness	0.060605	0.060416	0.060385	0.050140	0.049462
defocus_blur	mean_blue	0.038377	0.038431	0.038553	0.038591	0.038571
defocus_blur	mean_green	0.040674	0.040607	0.040626	0.040581	0.040763
defocus_blur	mean_red	0.059018	0.059035	0.059161	0.059179	0.059252
defocus_blur	rms_contrast	0.417192	0.486428	0.606822	0.669142	0.772384
defocus_blur	sharpness	1.265309	1.283204	1.292588	1.292072	1.294569
glass_blur	brightness	0.064491	0.064835	0.062973	0.063866	0.063666
glass_blur	mean_blue	0.040414	0.041017	0.039830	0.040450	0.040463
glass_blur	mean_green	0.040329	0.040112	0.039959	0.039883	0.039657
glass_blur	mean_red	0.060777	0.061247	0.060013	0.060569	0.060458
glass_blur	rms_contrast	0.350142	0.423292	0.497461	0.569429	0.691295
glass_blur	sharpness	1.158576	1.232642	1.238898	1.258086	1.286698
motion_blur	brightness	0.056901	0.056778	0.056420	0.055830	0.054923
motion_blur	mean_blue	0.034105	0.034367	0.034439	0.034397	0.034168
motion_blur	mean_green	0.041566	0.041327	0.041100	0.040844	0.040585
motion_blur	mean_red	0.055418	0.055543	0.055489	0.055315	0.054954
motion_blur	rms_contrast	0.344950	0.438142	0.551789	0.662401	0.744262
motion_blur	sharpness	1.041598	1.140697	1.199950	1.232650	1.247467
zoom_blur	brightness	0.066676	0.068524	0.070336	0.071692	0.073191
zoom_blur	mean_blue	0.043693	0.045843	0.047872	0.049723	0.051801
zoom_blur	mean_green	0.049571	0.052709	0.055854	0.058789	0.062629
zoom_blur	mean_red	0.068991	0.072782	0.076475	0.079874	0.083973
zoom_blur	rms_contrast	0.458881	0.526616	0.611801	0.661374	0.740796
zoom_blur	sharpness	1.165725	1.206743	1.218080	1.235984	1.237147

Tabella 5.2: Distanza di Wesserstein tra le distribuzioni delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-C nel gruppo di corruzione blur.

Possiamo notare dai dati in tabella 5.1.2 che per tutti i tipi di blur la brightness e i canali colore non driftano, questo accade perché i blur non modificano la luminosità o i colori delle immagini. Le feature rms_contrast e sharpness driftano a tutti i livelli di severità. In particolare, la feature sharpness presenta un drift molto più marcato rispetto alle altre feature, con valori di distanza di Wesserstein che superano 1.2 per la severità 5. Questo, può essere causato dal fatto che il blur può coprire i bordi degli oggetti, diminuendo la nitidezza e il contrasto, come si può osservare ad esempio nella figura 5.12. Come si può notare inoltre dalla figura 5.3, le feature che non presentano drift hanno un andamento pressoché costante per ogni livello di severità, tranne per zoom_blur, il quale andamento è quasi lineare all'aumentare del livello di severità. Inoltre, i valori della distanza di Wesserstein per sharpness sono molto vicini fra loro per ogni livello di severità e tipo di blur.

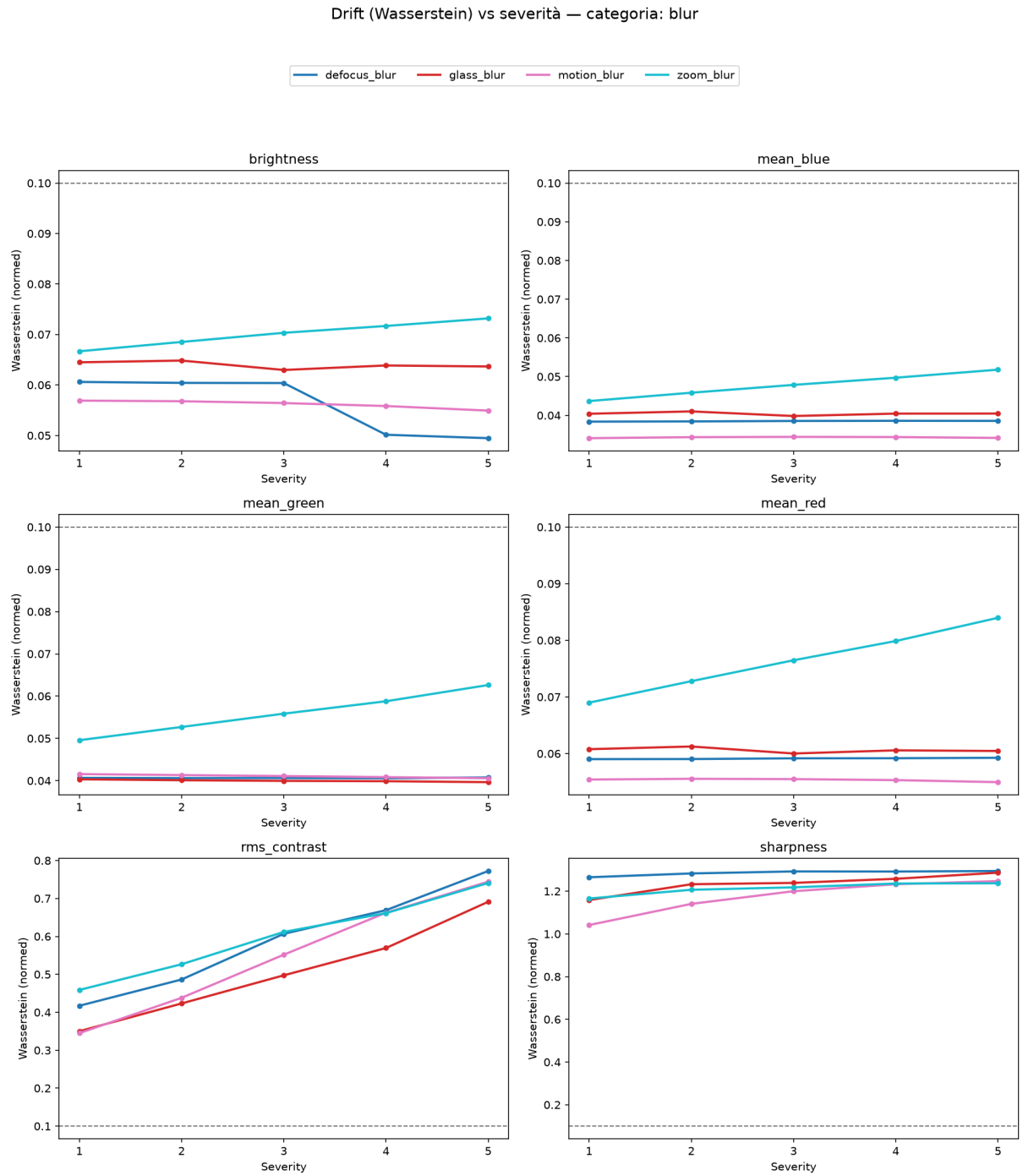


Figura 5.3: Distanza di Wesserstein tra le distribuzioni delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-C nel gruppo di corruzione blur, al variare della severità.

corruption	feature	sev_1	sev_2	sev_3	sev_4	sev_5
brightness	brightness	0.576325	1.128612	1.609895	2.022776	2.348483
brightness	mean_blue	0.037699	0.043582	0.047753	0.053588	0.056299
brightness	mean_green	0.039589	0.040811	0.042565	0.052990	0.059364
brightness	mean_red	0.057856	0.063008	0.064307	0.068695	0.069323
brightness	rms_contrast	0.194603	0.343004	0.586194	0.913852	1.269411
brightness	sharpness	0.239242	0.248618	0.285387	0.348412	0.429741
fog	brightness	0.337783	0.344840	0.364152	0.401109	0.441012
fog	mean_blue	0.525600	0.576151	0.612098	0.613082	0.639908
fog	mean_green	0.380592	0.415061	0.439368	0.440063	0.458478
fog	mean_red	0.457440	0.503187	0.535868	0.536735	0.561158
fog	rms_contrast	1.694664	1.723715	1.590665	1.471921	1.383768
fog	sharpness	1.103313	1.169476	1.208260	1.207460	1.229185
frost	brightness	1.681603	1.974560	2.112238	1.987928	2.055561
frost	mean_blue	0.469288	0.643080	0.722371	0.743677	0.783210
frost	mean_green	0.212773	0.240710	0.249700	0.248716	0.254829
frost	mean_red	0.436628	0.609805	0.691184	0.713028	0.753416
frost	rms_contrast	0.537042	1.067577	1.318931	1.377810	1.495522
frost	sharpness	0.285483	0.293777	0.247972	0.242873	0.210797
snow	brightness	1.146456	1.850992	1.839177	2.211917	2.592699
snow	mean_blue	0.373648	0.528322	0.529197	0.599734	0.670009
snow	mean_green	0.269782	0.371856	0.373095	0.420665	0.467768
snow	mean_red	0.322548	0.463436	0.463859	0.526932	0.588320
snow	rms_contrast	0.099864	0.189850	0.176810	0.226027	0.652584
snow	sharpness	0.461836	1.037702	0.549788	0.515835	0.599181

Tabella 5.3: Distanza di Wesserstein tra le distribuzioni delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-C nel gruppo di corruzione weather.

Come si può notare dalla tabella 5.1.2, per il tipo di sporcatrice brightness, i valori medi dei canali colore non presentano drift, poiché modificare la luminosità non comporta una modifica nel colore dell'immagine, mentre le feature brightness, rms_contrast e sharpness invece presentano un drift rispetto alla distribuzione di confronto. Infatti, come si può notare dall'immagine 5.16 aumentando la luminosità dell'immagine, oltre ovviamente ad introdurre un drift nella distribuzione della feature brightness, la feature contrast presenta una diminuzione dei suoi valori mentre la feature sharpness presenta una minore concentrazione nelle code della distribuzione, mentre aumenta la frequenza dei valori più frequenti, portando le distribuzioni a schiacciarsi. Questo può essere dal fatto che aumentare la luminosità di un'immagine la porta ad essere più tendente al bianco, quindi diminuendo il contrasto e mediamente diminuendo la sua nitidezza. Possiamo inoltre fare le seguenti affermazioni riguardanti la figura 5.4:

- La distanza di Wesserstein per le immagini sporcate con corruzione snow ha un picco con severity 2 per la feature sharpness. Infatti, come si può notare dall'immagine 5.15, i valori di sharpness per severity 2 sono molto più alti rispetto alla distribuzione originale e alle distribuzioni delle altre severity. Questo può essere dato dal fatto che aumentando il numero di oggetti nell'immagine senza però andandola a coprire del tutto, i valori della nitidezza aumentano poiché anche gli elementi introdotti dalla sporcatrice hanno bordi ben definiti.
- La distanza di Wesserstein per le immagini sporcate con corruzione fog diminuisce dalla severity 2 in poi per la feature rms_contrast. Come si può infatti notare dalla figura 5.14, la distribuzione di rms_contrast per le immagini sporcate presentano una minore variabilità e si concentrano su minori valori di contrasto. Questo può essere dato dal fatto che applicando questo tipo di sporcatrice, le immagini tendono verso una colorazione grigia uniforme, come si può anche notare dalle distribuzioni dei canali colore.

Drift (Wasserstein) vs severità — categoria: weather

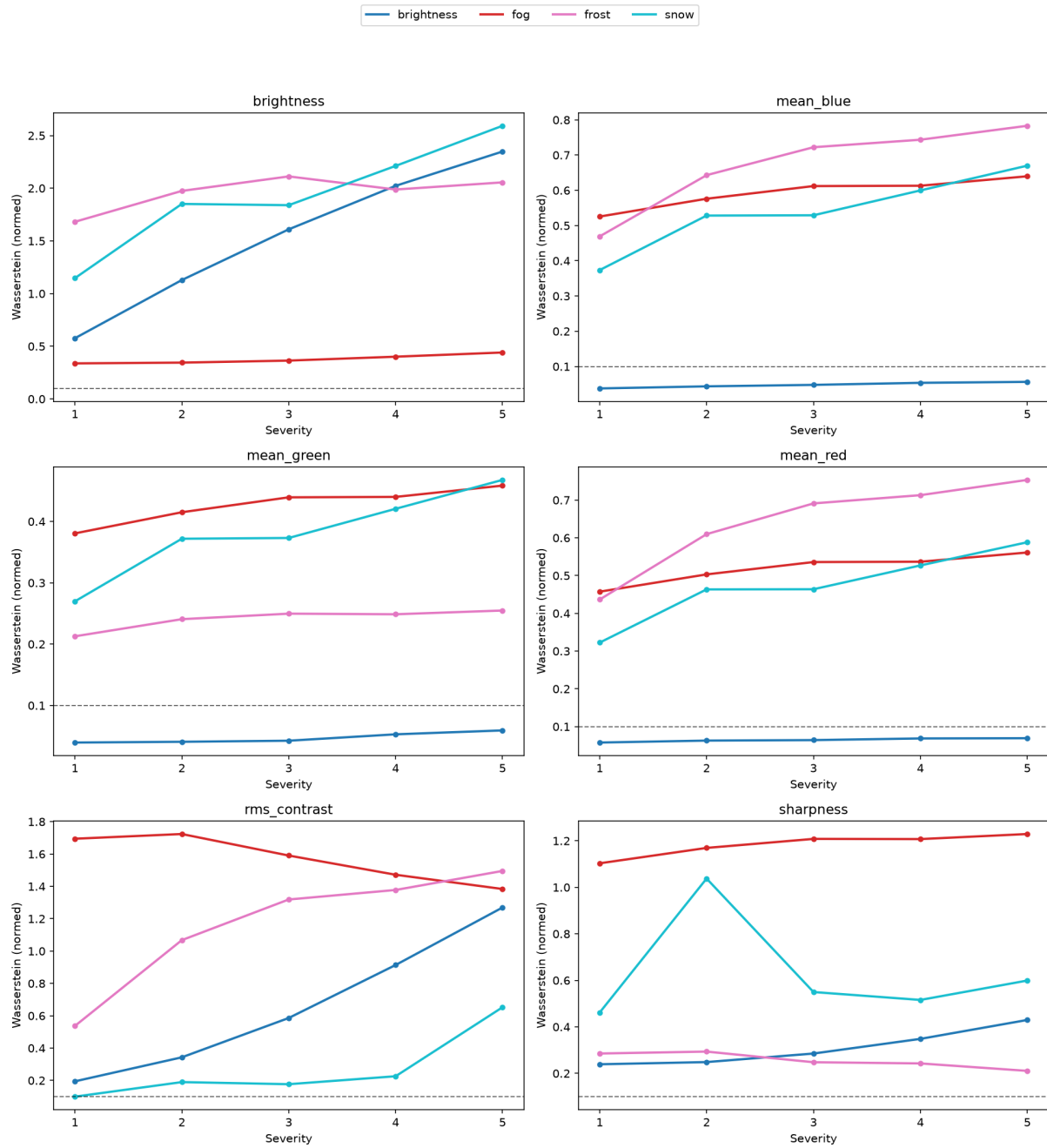


Figura 5.4: Distanza di Wesserstein tra le distribuzioni delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-C nel gruppo di corruzione weather, al variare della severità.

corruption	feature	sev_1	sev_2	sev_3	sev_4	sev_5
contrast	brightness	0.060245	0.060239	0.060229	0.060227	0.060195
contrast	mean_blue	0.038418	0.038245	0.037904	0.037598	0.037547
contrast	mean_green	0.041440	0.041902	0.042790	0.043391	0.043403
contrast	mean_red	0.059435	0.059512	0.059563	0.059451	0.059261
contrast	rms_contrast	2.330083	2.691652	3.053031	3.413698	3.593123
contrast	sharpness	1.112014	1.212285	1.281598	1.318082	1.325120
elastic_transform	brightness	0.051794	0.044459	0.059603	0.059610	0.059654
elastic_transform	mean_blue	0.039154	0.042950	0.036830	0.036657	0.036251
elastic_transform	mean_green	0.043501	0.049631	0.041769	0.041790	0.042117
elastic_transform	mean_red	0.061286	0.067652	0.058096	0.057926	0.057702
elastic_transform	rms_contrast	0.269896	0.304468	0.249420	0.249590	0.249708
elastic_transform	sharpness	0.862587	0.768063	0.821826	0.774113	0.614177
jpeg_compression	brightness	0.057809	0.057875	0.058283	0.058583	0.057985
jpeg_compression	mean_blue	0.022065	0.026881	0.018877	0.021624	0.023145
jpeg_compression	mean_green	0.060105	0.051337	0.057659	0.054069	0.058407
jpeg_compression	mean_red	0.052134	0.051353	0.046059	0.043821	0.035850
jpeg_compression	rms_contrast	0.183292	0.187883	0.190634	0.196080	0.196912
jpeg_compression	sharpness	0.469067	0.514780	0.534496	0.564141	0.578935
pixelate	brightness	0.056300	0.056056	0.056462	0.056612	0.056343
pixelate	mean_blue	0.029980	0.029781	0.031634	0.032457	0.032972
pixelate	mean_green	0.043141	0.042790	0.042617	0.042332	0.041621
pixelate	mean_red	0.052299	0.051931	0.053595	0.054230	0.054373
pixelate	rms_contrast	0.223815	0.236370	0.276135	0.321615	0.353021
pixelate	sharpness	0.318243	0.342908	0.411007	0.488771	0.533425

Tabella 5.4: Distanza di Wesserstein tra le distribuzioni delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-C nel gruppo di corruzione digital.

Come si può notare dalla tabella 5.1.2, le uniche feature che presentano un drift secondo la distanza di Wesserstein sono il contrasto e la nitidezza. In particolare possiamo compiere le seguenti osservazioni:

- Nonostante venga modificato direttamente il contrasto dell'immagini nel tipo di sporcatrice contrast, la media dei canali colori non presenta drift in nessun livello di severità. Infatti, come si può osservare nella figura 5.17, nonostante il contrasto venga diminuito rendendo l'immagine più uniforme a livello di colore, il valore medio dei canali colore rimane sostanzialmente invariato.

Drift (Wasserstein) vs severità — categoria: digital

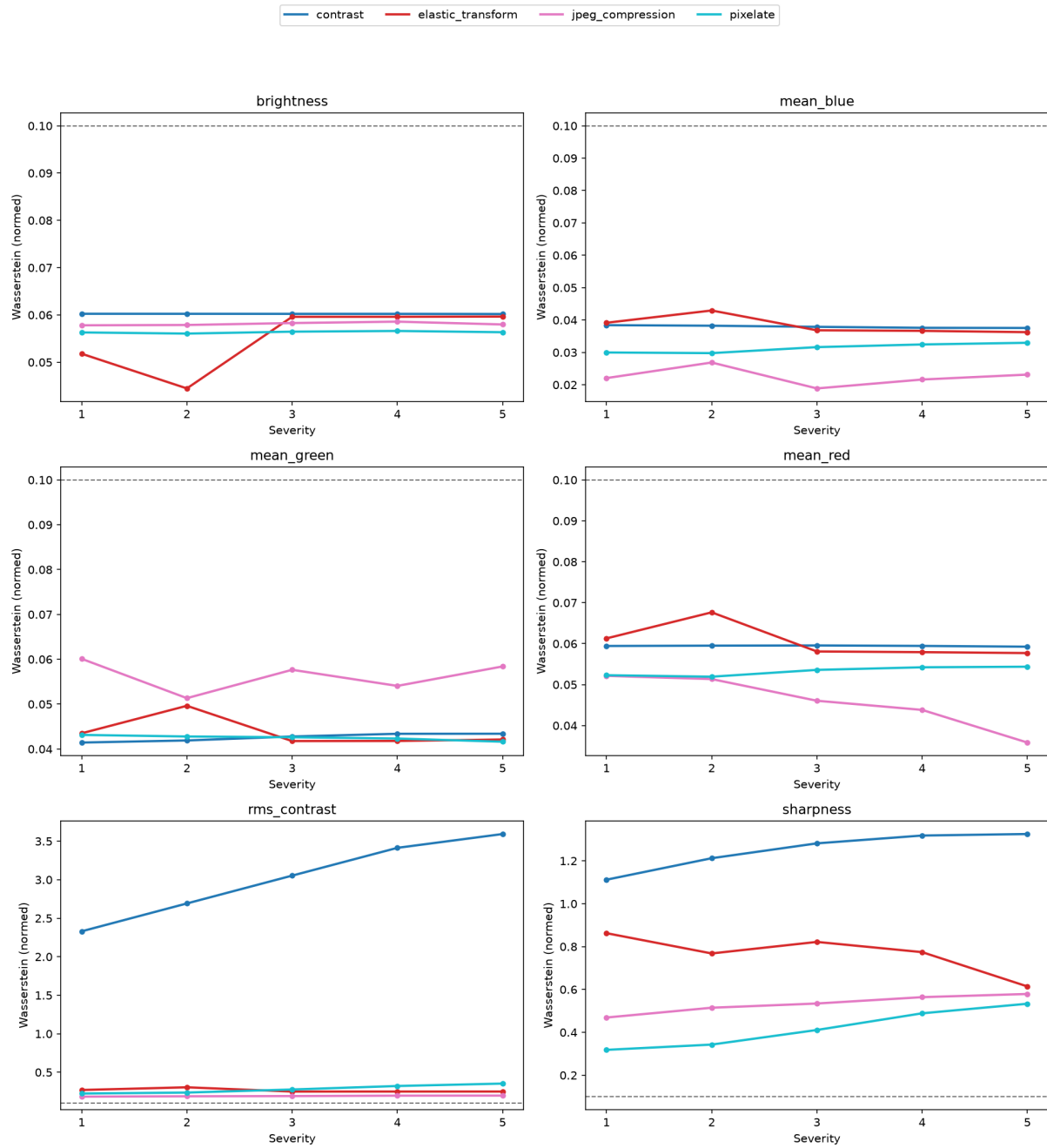


Figura 5.5: Distanza di Wesserstein tra le distribuzioni delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-C nel gruppo di corruzione digital, al variare della severità.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: gaussian_noise

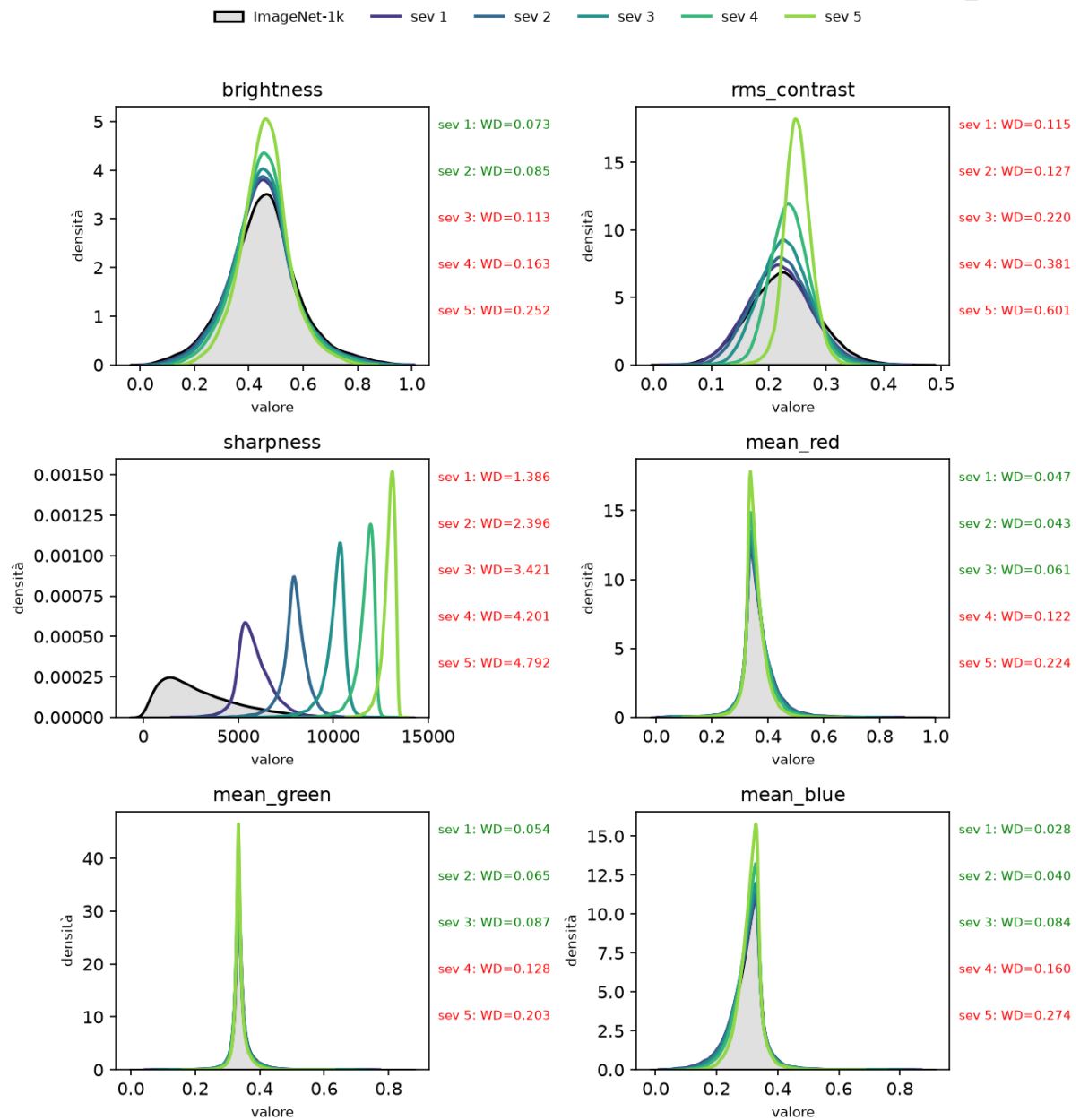


Figura 5.6: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con rumore gaussiano.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: shot_noise

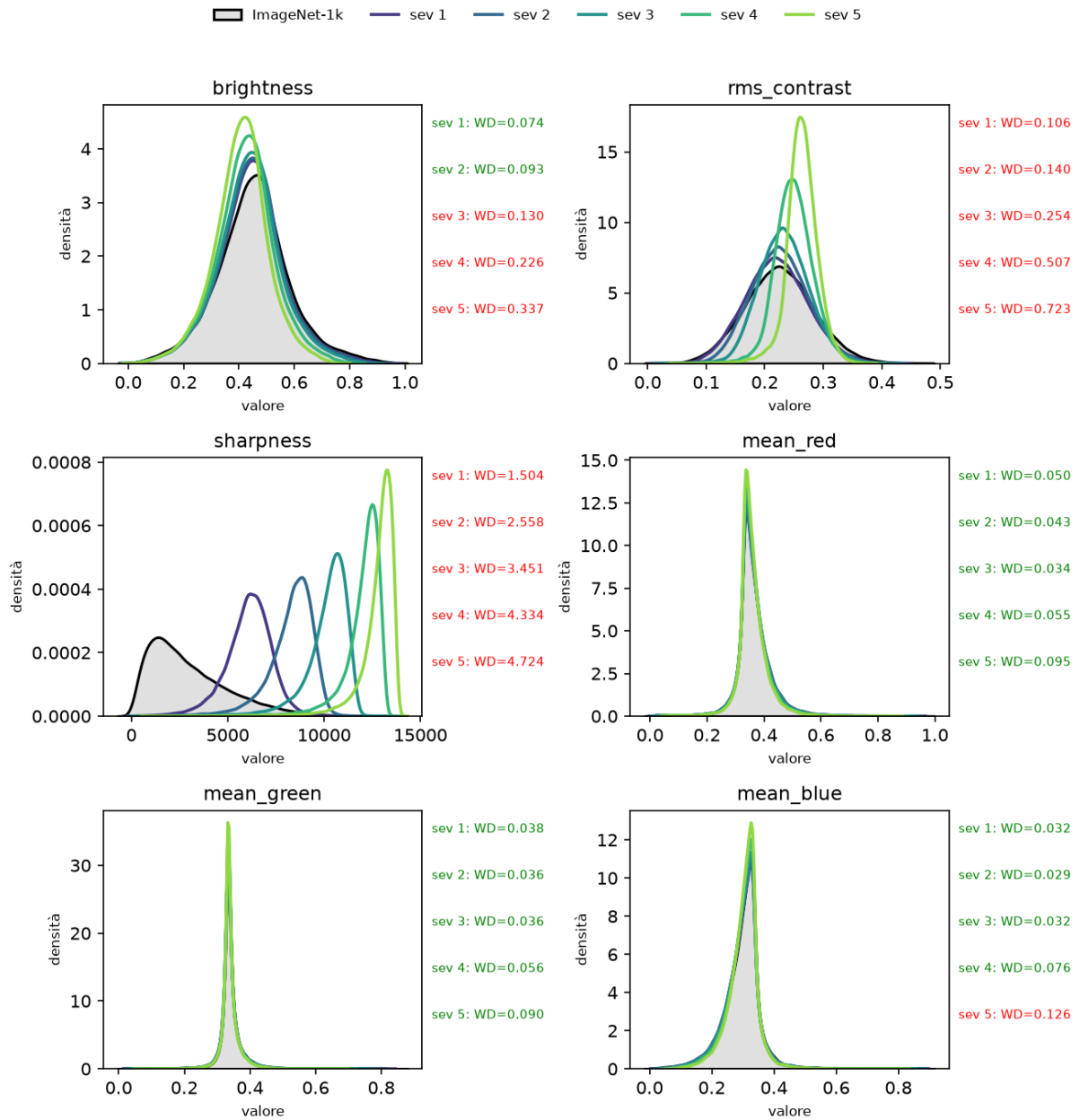


Figura 5.7: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con rumore di shot.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: impulse_noise

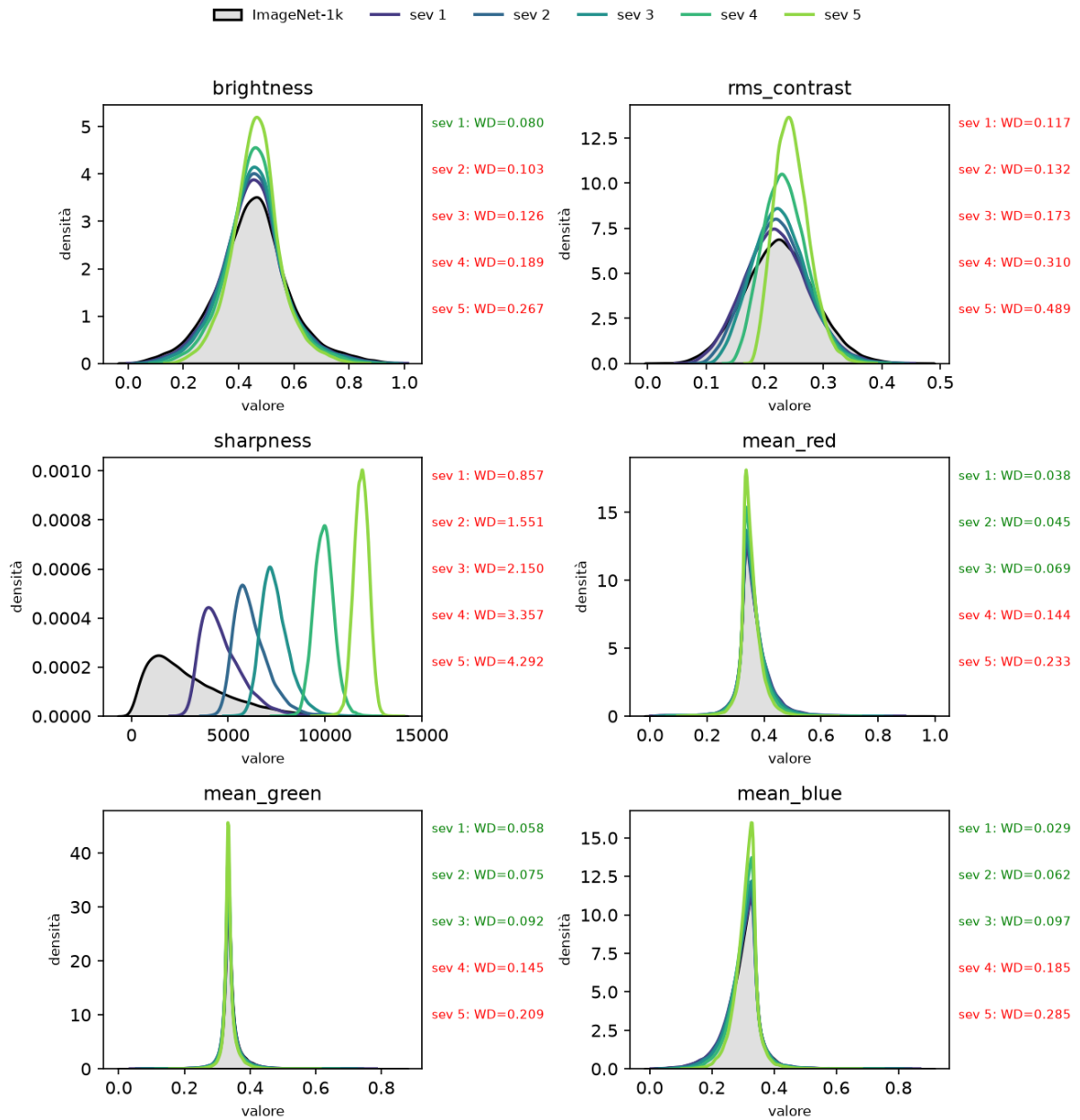


Figura 5.8: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con rumore impulsivo.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: defocus_blur

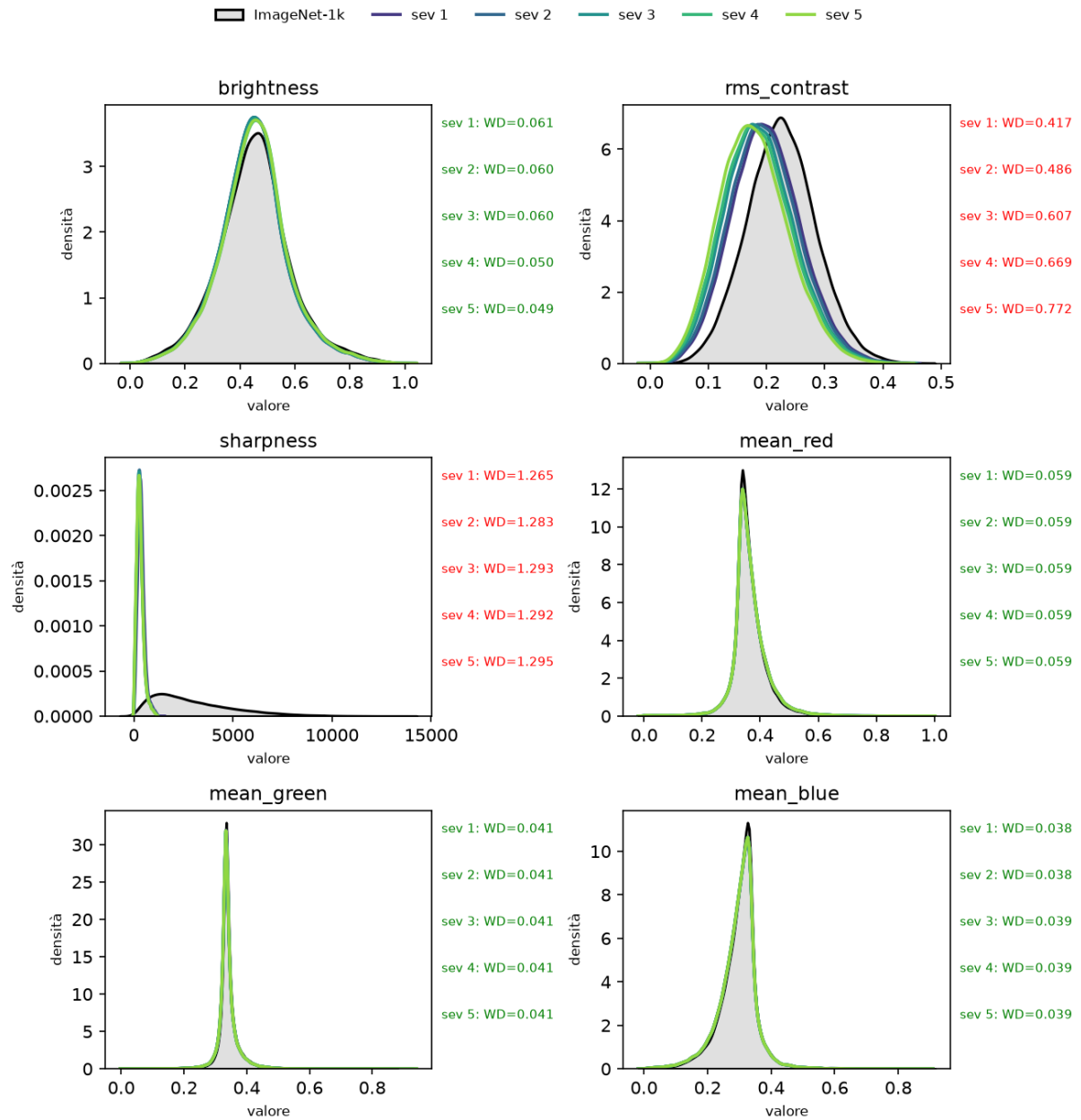


Figura 5.9: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con sfocatura da defocus.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: motion_blur

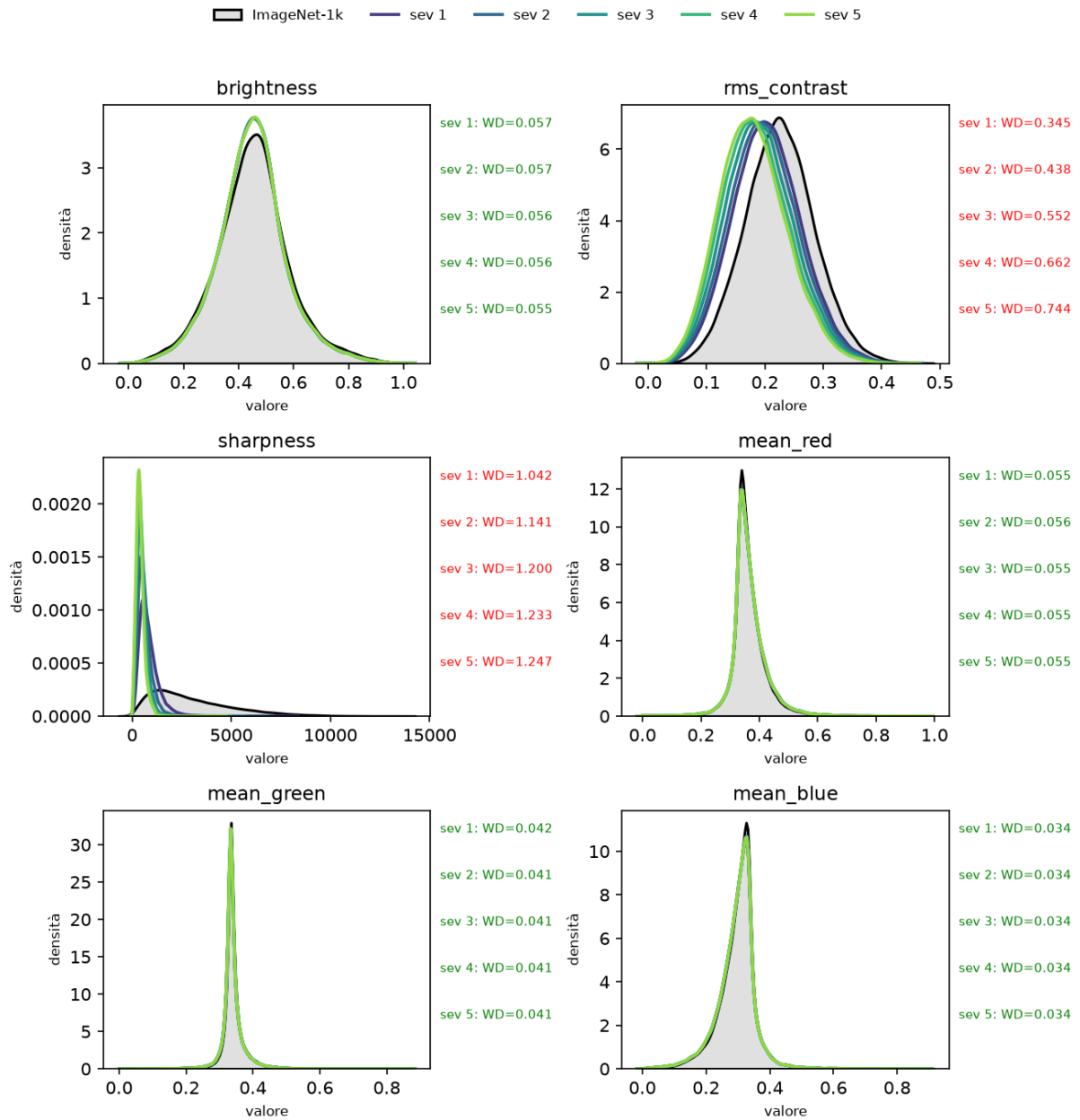


Figura 5.10: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con sfocatura da movimento.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: zoom_blur

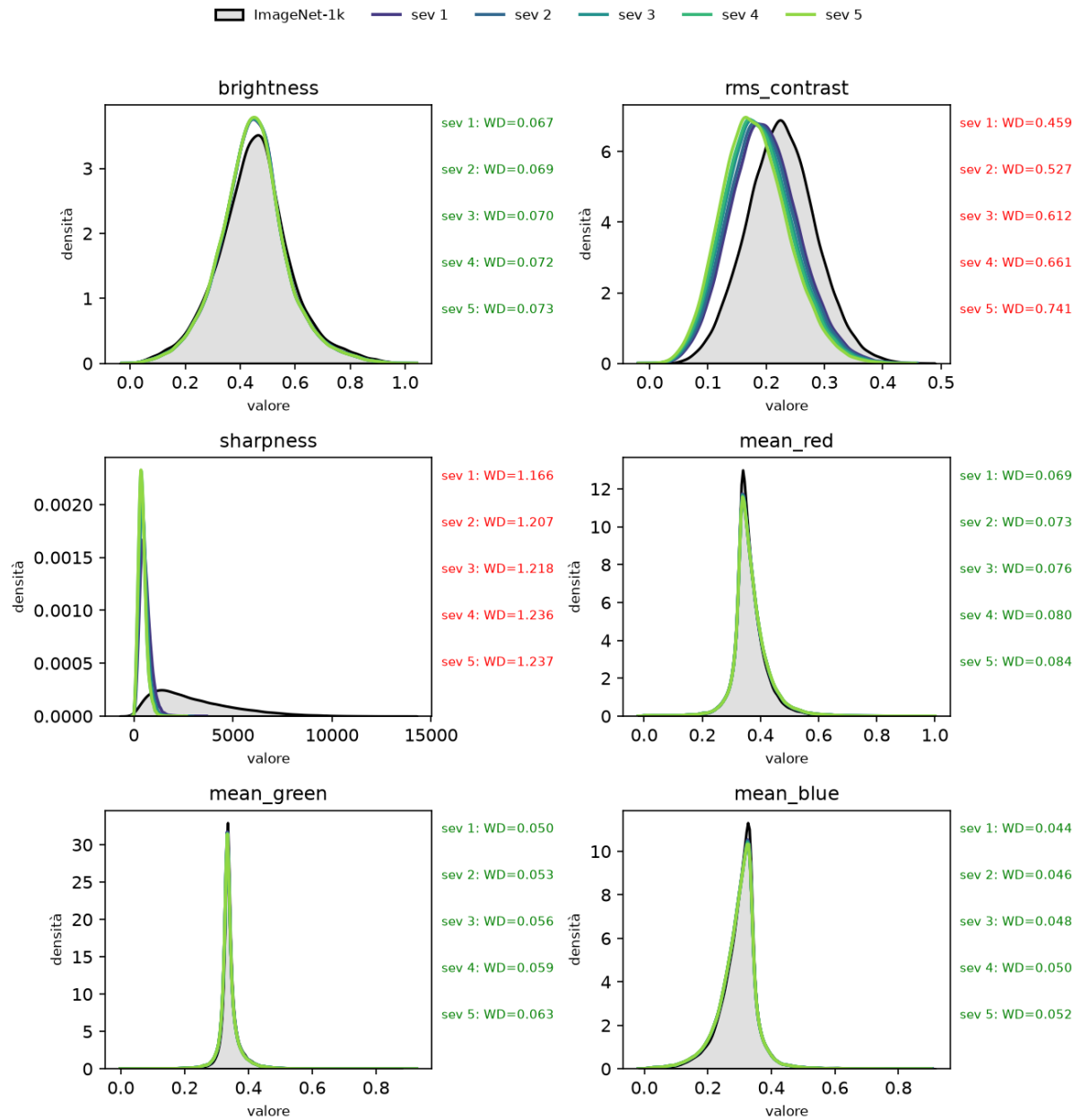


Figura 5.11: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con sfocatura da zoom.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: glass_blur

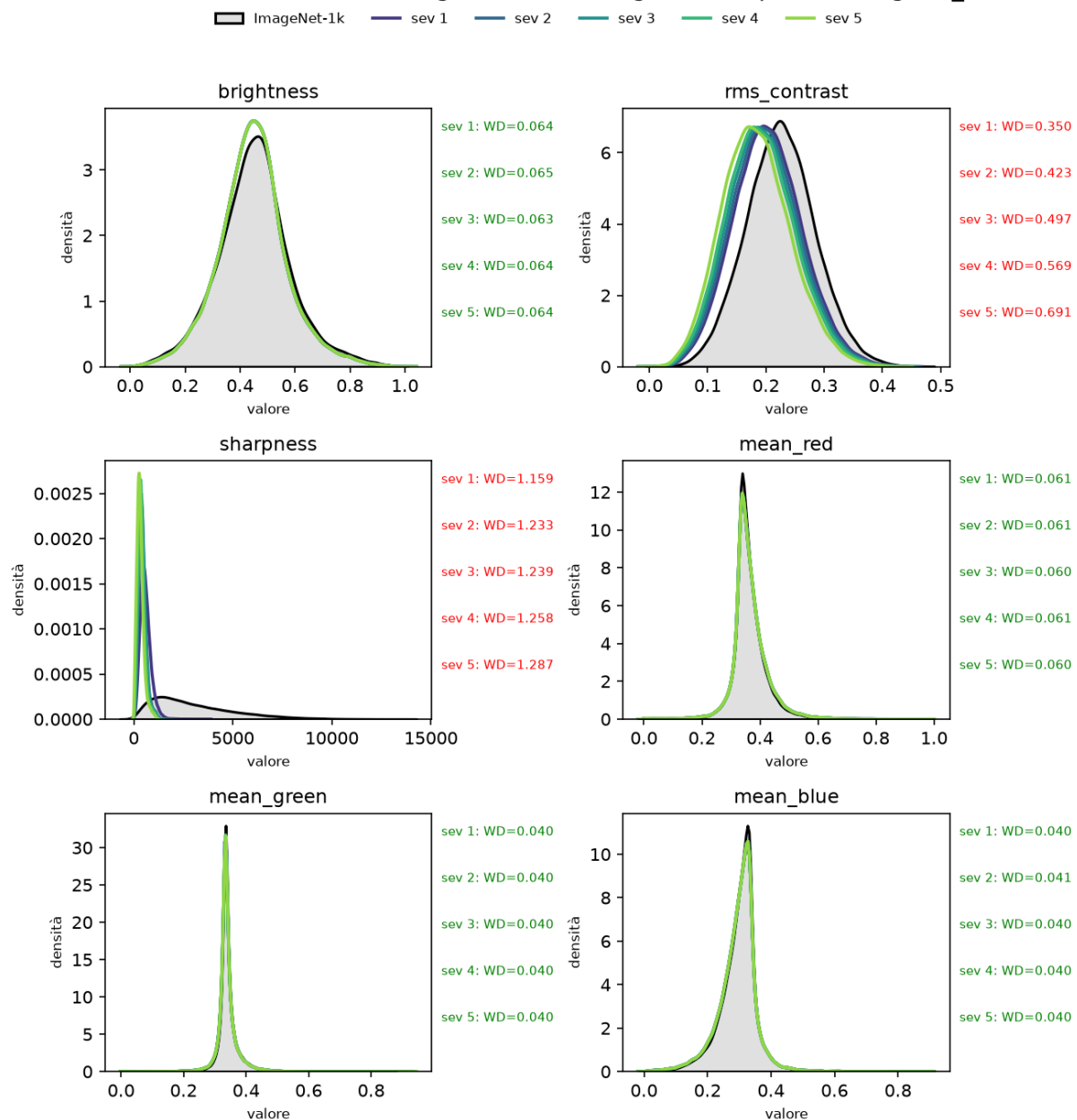


Figura 5.12: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con corruzione da vetro satinato.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: frost

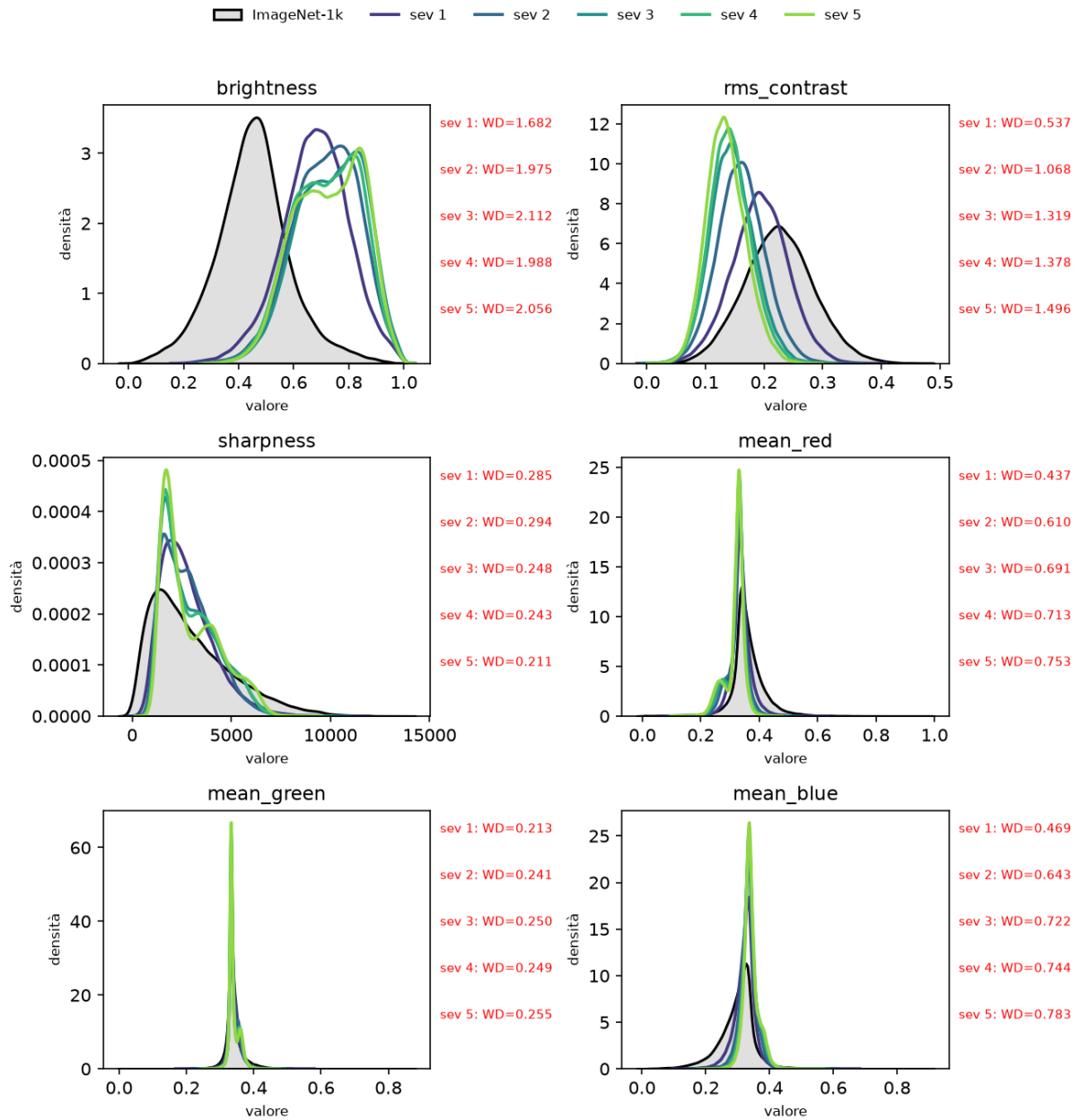


Figura 5.13: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con corruzione da brina.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: fog

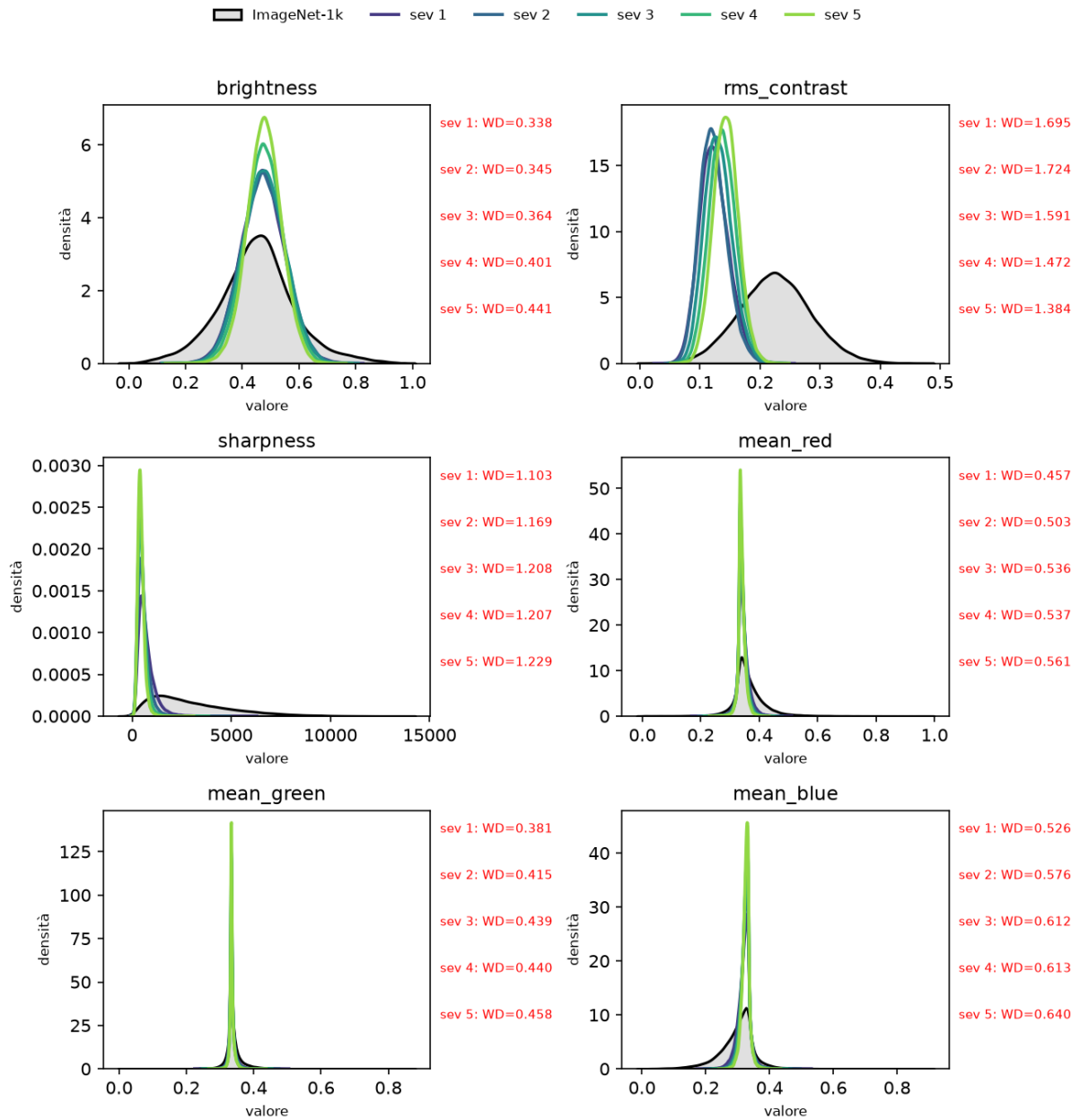


Figura 5.14: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con corruzione da nebbia.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: snow

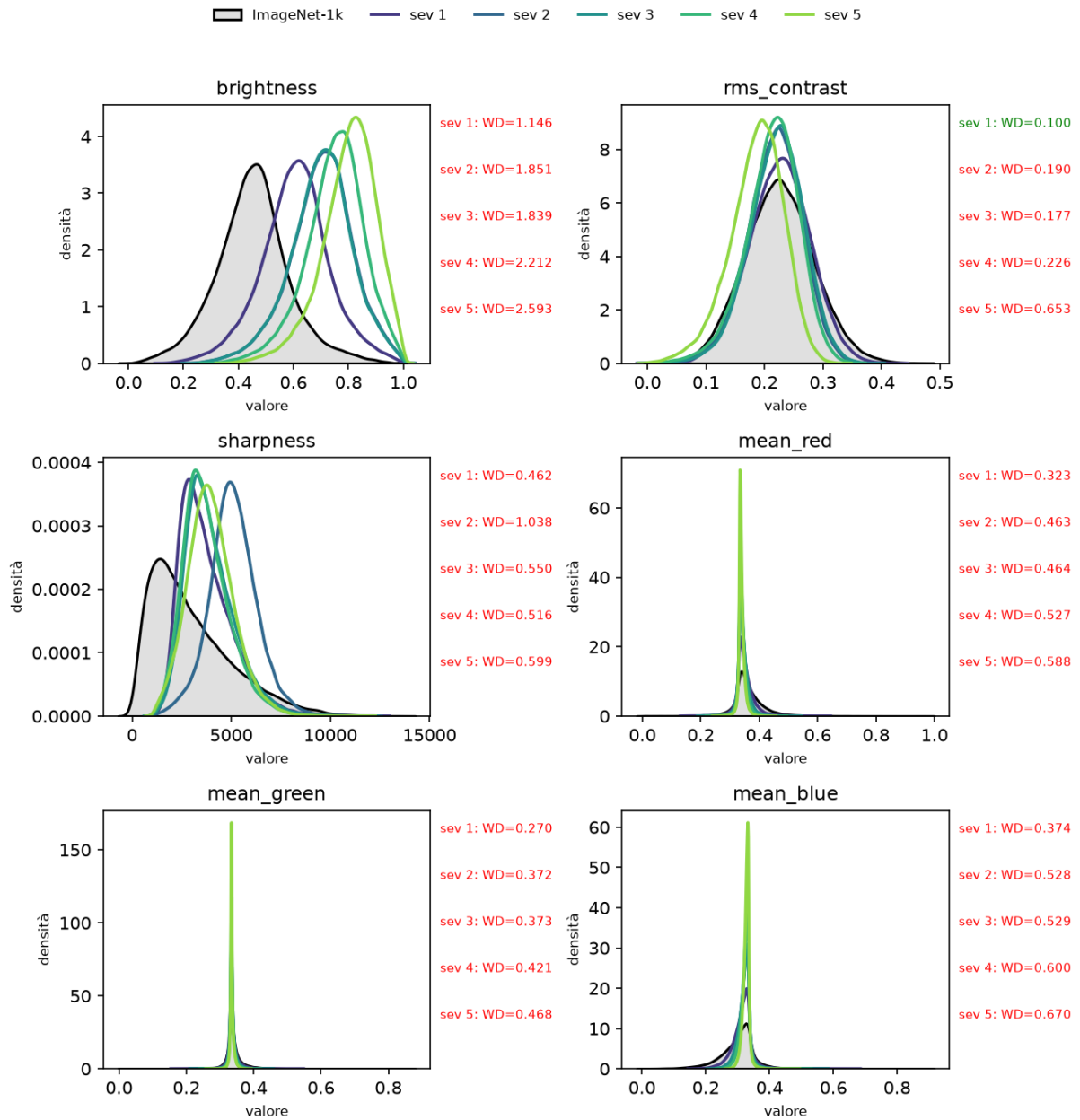


Figura 5.15: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con corruzione da neve.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: brightness

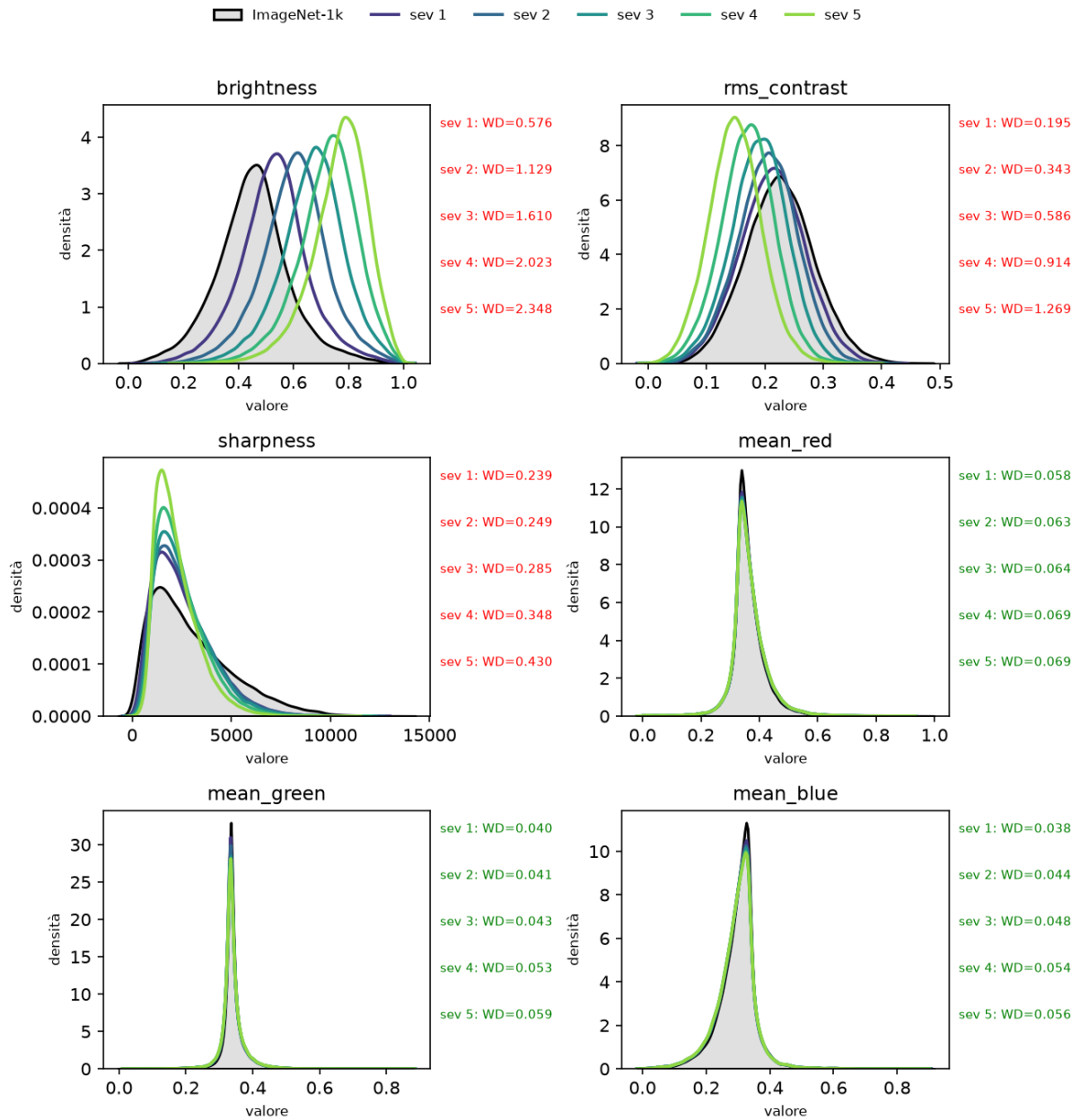


Figura 5.16: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con corruzione da luminosità.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: contrast

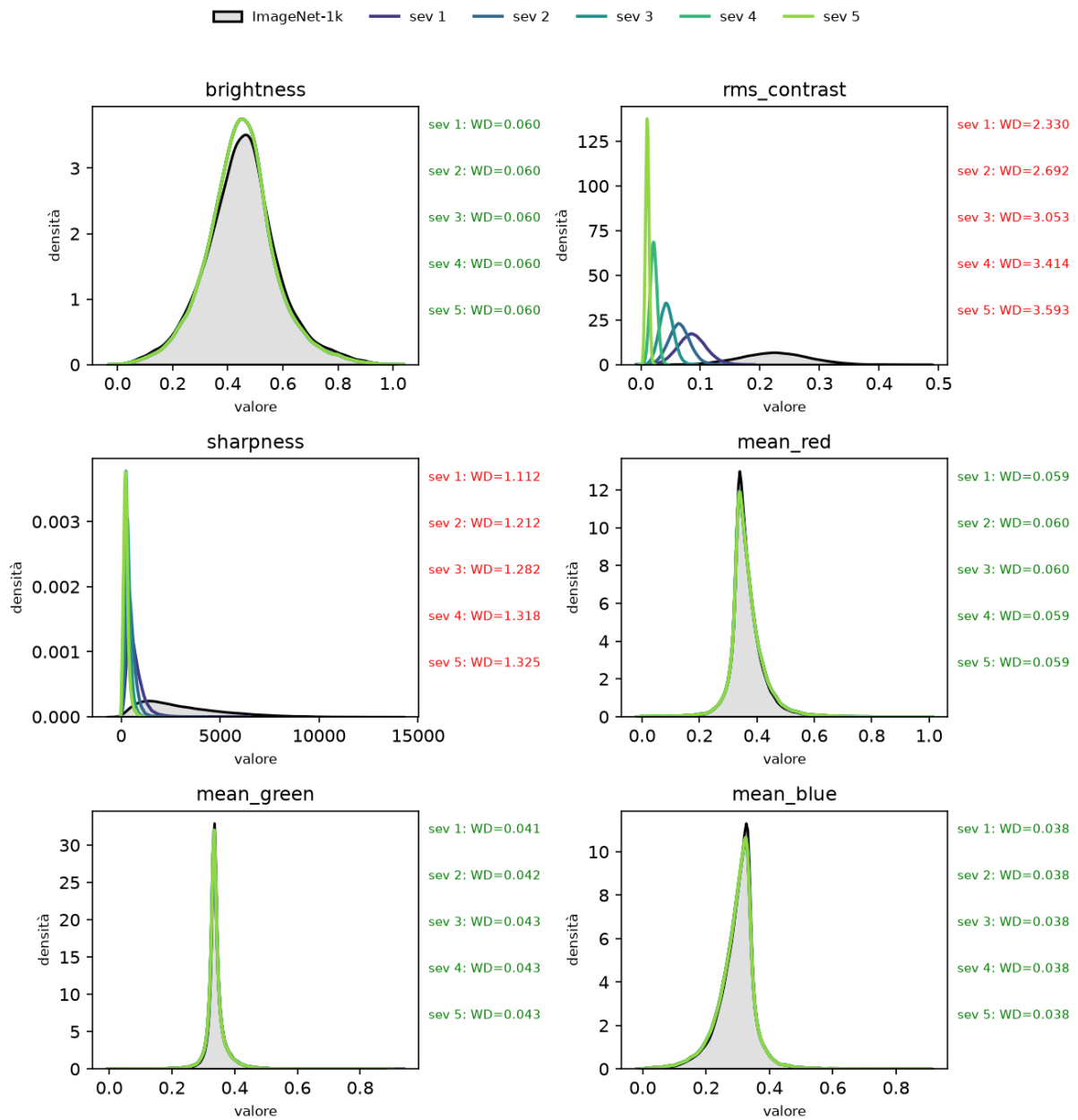


Figura 5.17: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con corruzione da contrasto.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: pixelate

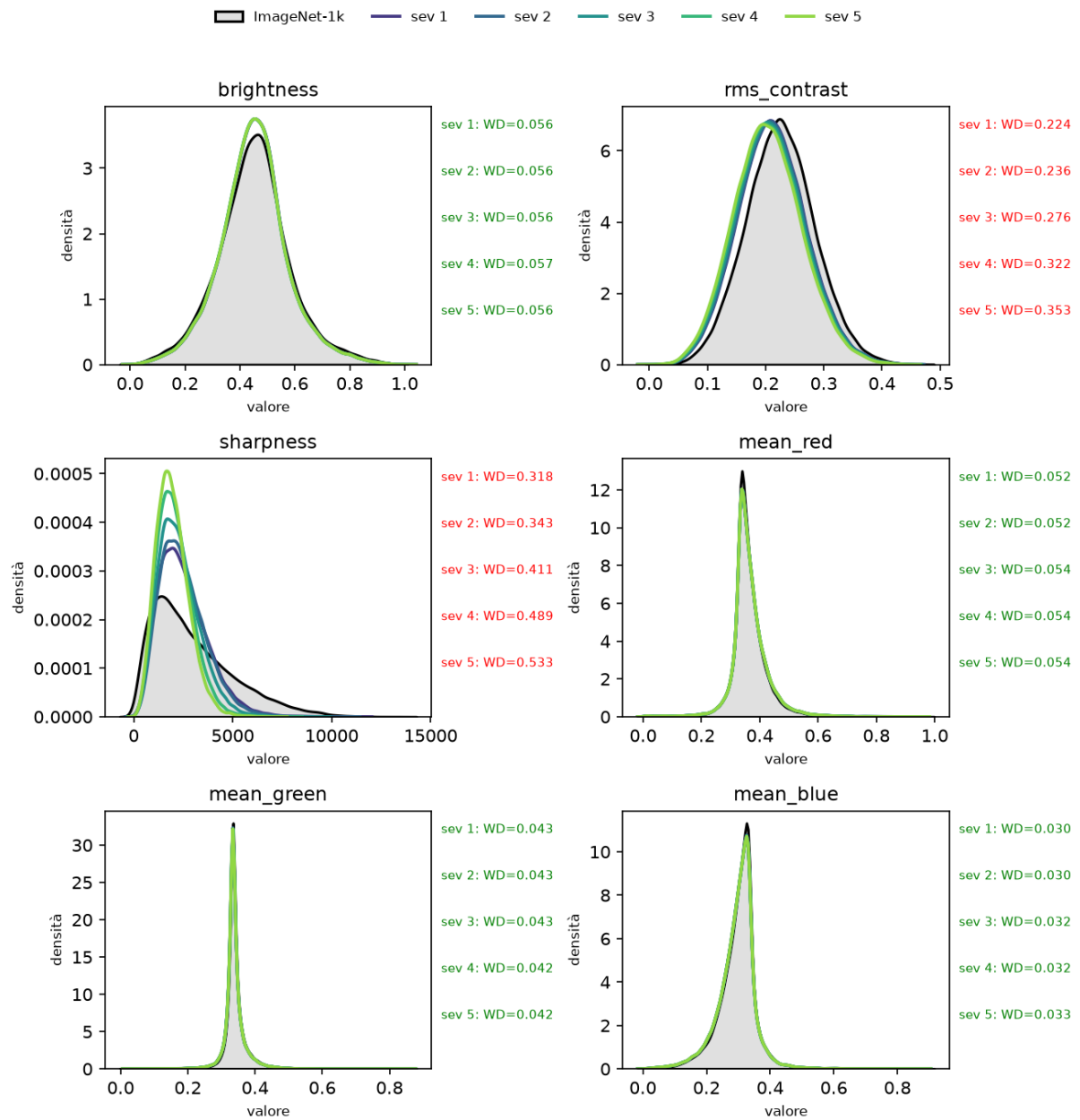


Figura 5.18: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con corruzione da pixelizzazione.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: jpeg_compression

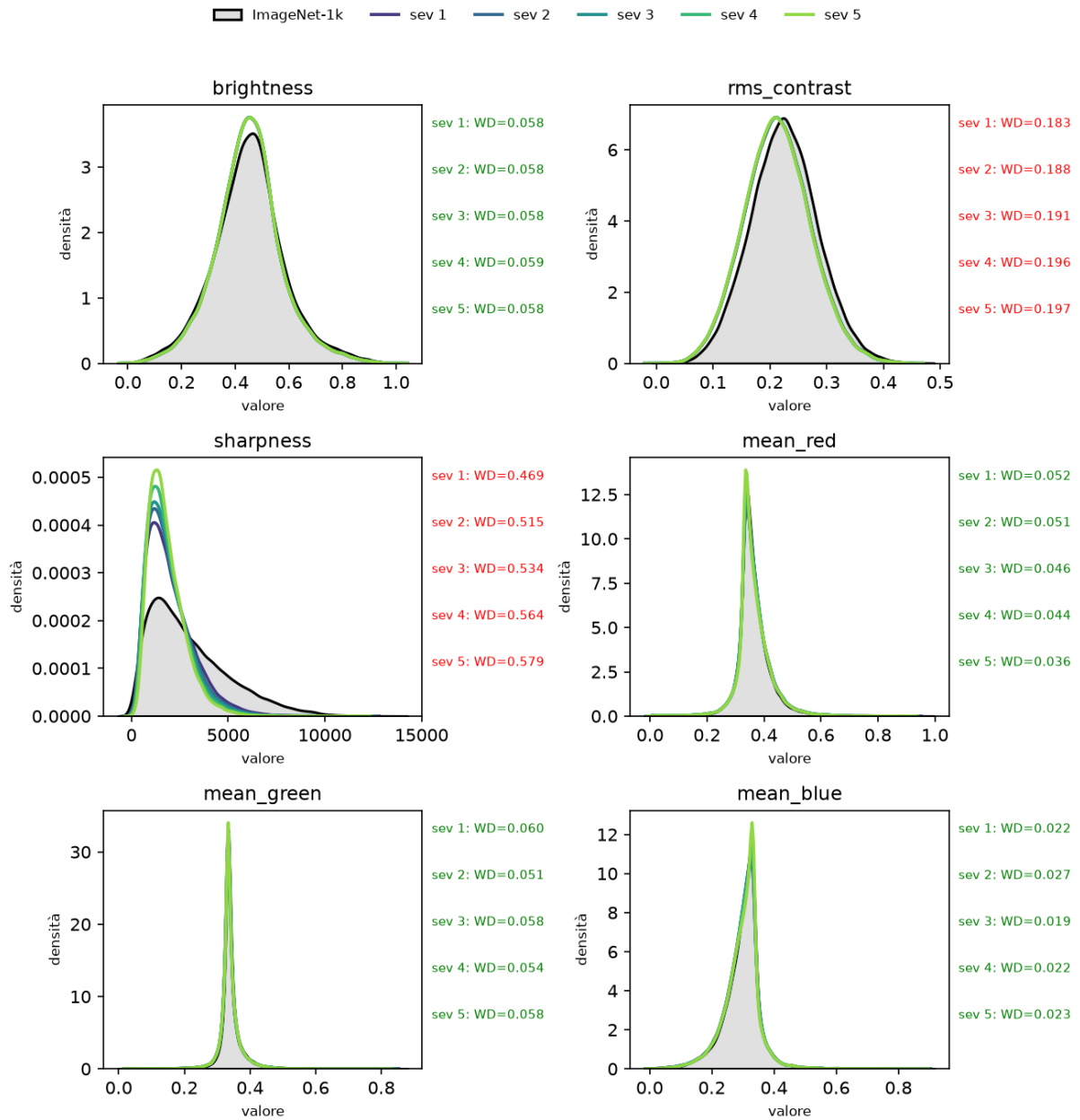


Figura 5.19: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con corruzione da compressione JPEG.

Distribuzioni delle feature: ImageNet-1k vs ImageNet-C. Sporcatura: elastic_transform

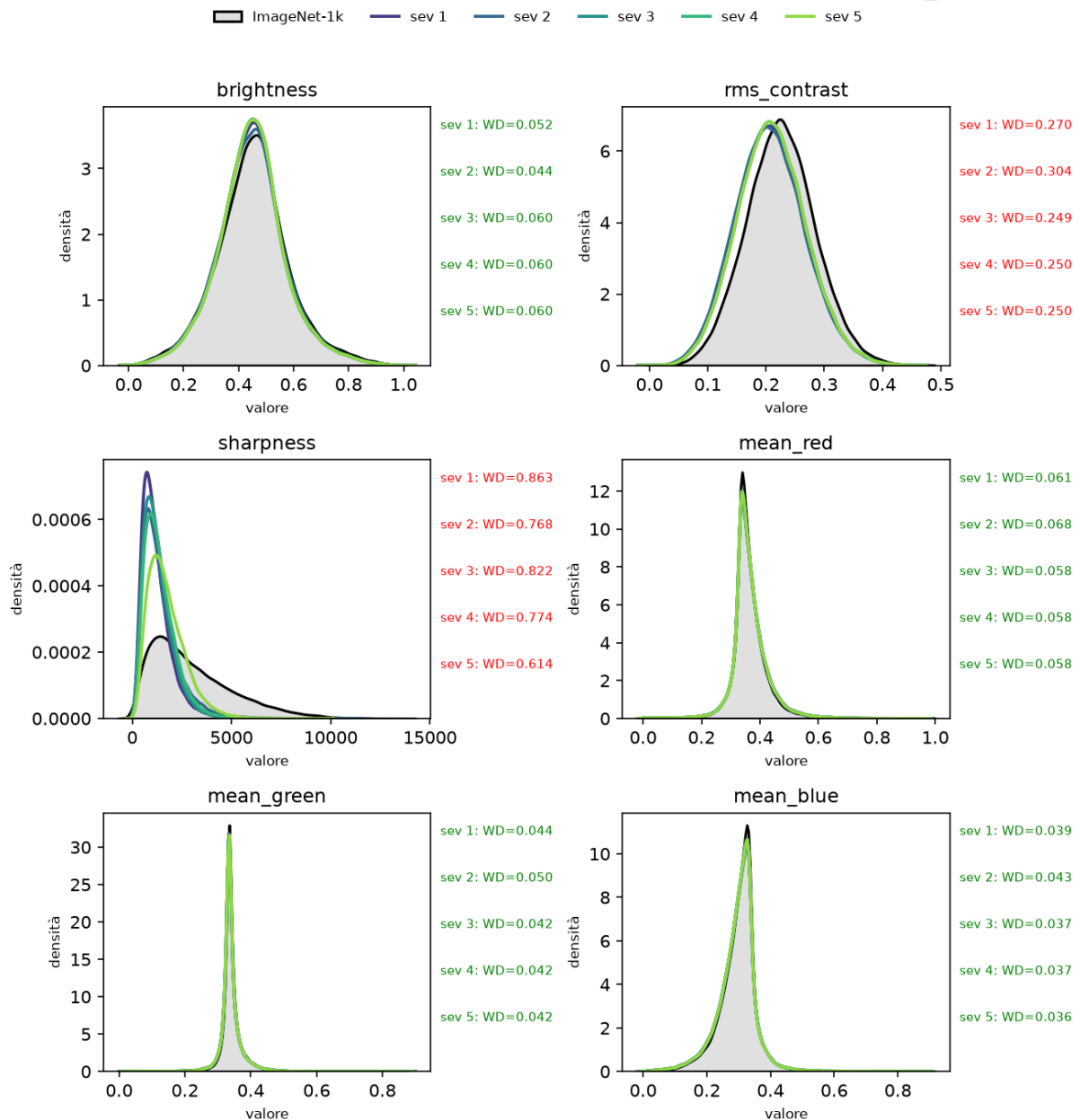


Figura 5.20: Distribuzione delle misure di luminanza, contrasto, nitidezza e media dei canali colore per ImageNet-1k e ImageNet-C con corruzione da trasformazione elastica.

5.2 Analisi delle metriche di drift sulle feature del modello

In questa sezione vengono analizzate le metriche di drift calcolate sulle feature del modello ResNet-50, estratte dal layer `avgpool1`. Come già accennato nella sezione 4, la rilevazione delle feature viene effettuata tramite test MMD. I test sono stati effettuati con un valore di soglia uguale a 0.05, quindi viene rilevato un drift quando il p-value $p < 0.05$.

5.2.1 ImageNet-1k vs ImageNet-R

Il test MMD ImageNet-R è stato effettuato rispetto alle feature ottenute dalla classificazione di ImageNet-1k sottocampionato alle duecento classi in comune con ImageNet-R.

dataset	is_drift	p_val	distance
ImageNetR	True	0.0000	0.0805

Tabella 5.5: Risultati del test di MMD tra le feature estratte dal layer **avgpool** del modello ResNet-50 per ImageNet-1k e ImageNet-R.

Come si può notare dal risultato riportato in tabella 5.5, il p-value del test è zero, quindi al di sotto della soglia di rilevazione. La rilevazione del drift è quindi coerente con il drift rilevato per le feature visive, come si può vedere in figura 5.1.

5.2.2 ImageNet-1k vs ImageNet-C

Per poter fare l'analisi di MMD delle feature, è stato necessario effettuare un campionamento a 10.000 esempi, in quanto utilizzare l'intero dataset avrebbe richiesto una quantità di memoria della GPU a noi non disponibile. Il campionamento è casuale, mantenendo un numero equo di esempi per ogni classe.

Analizziamo ora le metriche di drift rilevate nelle feature utilizzate dal modello durante la classificazione e confrontiamo il risultato con il drift rilevato nelle feature visive.

Blur

corruption	severity	is_drift	p_val	distance
defocus_blur	1	True	0.0000	0.0002
defocus_blur	2	True	0.0000	0.0003
defocus_blur	3	True	0.0000	0.0005
defocus_blur	4	True	0.0000	0.0009
defocus_blur	5	True	0.0000	0.0022
glass_blur	1	True	0.0000	0.0002
glass_blur	2	True	0.0000	0.0031
glass_blur	3	True	0.0000	0.0176
glass_blur	4	True	0.0000	0.0321
glass_blur	5	True	0.0000	0.0238
motion_blur	1	False	0.2000	0.0000
motion_blur	2	False	0.1500	0.0001
motion_blur	3	True	0.0000	0.0013
motion_blur	4	True	0.0000	0.0099
motion_blur	5	True	0.0000	0.0229
zoom_blur	1	True	0.0000	0.0006
zoom_blur	2	True	0.0000	0.0021
zoom_blur	3	True	0.0000	0.0017
zoom_blur	4	True	0.0000	0.0027
zoom_blur	5	True	0.0000	0.0013

Tabella 5.6: Risultati del test di MMD tra le feature estratte dal layer **avgpool** del modello ResNet-50 per ImageNet-1k e ImageNet-C, corruzione di tipo blur.

Per quanto riguarda la classe di sporcatore blur, abbiamo ottenuto i risultati presentati in tabella 5.6. Possiamo effettuare le seguenti osservazioni:

- **defocus_blur**: Il drift viene rilevato ad ogni livello di severity; il che è coerente con il drift rilevato nelle feature visive, come presentato in figura 5.9. Infatti, in questo tipo di sporcatore sharpness e rms_contrast presentano un drift a tutti i livelli di severity.
- **glass_blur**: Come si può notare in figura 5.12, le feature sharpness e rms_contrast presentano un drift a tutti i livelli di severity. Valgono quindi le stesse considerazioni effettuate per defocus_blur.
- **motion_blur**: Nonostante venga rilevato, come nelle due sporcatore precedenti e riportato in figura 5.10, un drift nelle feature sharpness e rms_contrast in tutti i livelli di severità, con distanze di Wessersstein simili a quelle rilevate per defocus_blur e glass_blur, le feature estratte dal modello

per i primi due livelli non presentano drift rispetto alle feature estratte da ImageNet-1k. Infatti, le feature che il modello estrae e utilizza per la classificazione sono fondamentalmente diverse rispetto a quelle estraibili direttamente dalle immagini, quindi è possibile che, nonostante la sporcatura, le feature estratte per i primi due livelli di severità non presentino drift rispetto a quelle estratte per imagenet-1k.

- **zoom_blur:** Valgono le stesse considerazioni effettuate per defocus_blur e glass_blur. Ciò si può notare osservando la figura 5.11.

Digital

corruption	severity	is_drift	p_val	distance
contrast	1	False	0.2500	0.0000
contrast	2	False	0.1500	0.0001
contrast	3	False	0.1500	0.0001
contrast	4	True	0.0000	0.0005
contrast	5	True	0.0000	0.0029
elastic_transform	1	False	0.2500	0.0001
elastic_transform	2	True	0.0000	0.0011
elastic_transform	3	True	0.0000	0.0017
elastic_transform	4	True	0.0000	0.0083
elastic_transform	5	True	0.0000	0.0740
jpeg_compression	1	False	0.0500	0.0001
jpeg_compression	2	False	0.1500	0.0001
jpeg_compression	3	False	0.1000	0.0002
jpeg_compression	4	False	0.0500	0.0002
jpeg_compression	5	False	0.0500	0.0002
pixelate	1	False	0.2000	0.0001
pixelate	2	True	0.0000	0.0002
pixelate	3	True	0.0000	0.0005
pixelate	4	True	0.0000	0.0020
pixelate	5	True	0.0000	0.0126

Tabella 5.7: Risultati del test di MMD tra le feature estratte dal layer `avgpool` del modello ResNet-50 per ImageNet-1k e ImageNet-C, corruzione di tipo digital.

Per quanto riguarda la classe di sporcature digital, abbiamo ottenuto i risultati presentati in tabella 5.7. Possiamo effettuare le seguenti osservazioni:

- **contrast:** A differenza di quanto rilevato per le feature visive `rms_contrast` e `sharpness`, come si può vedere in figura 5.17, il test MMD non rileva una presenza di drift nelle feature ai primi tre livelli di severità. Valgono quindi le stesse considerazioni effettuate per `motion_blur`.
- **elastic_transform:** Come rilevato per `contrast`, vi è una discrepanza fra il drift rilevato nelle feature visive, riportato in figura 5.20, e il drift rilevato nelle feature estratte dal modello. In particolare, esse non risultano soggette a drift per il primo livello di severity.
- **jpeg_compression:** Per questo tipo di sporcatura, le feature estratte dal modello non risultano soggette a drift per nessun livello di severity, nonostante le feature visive `rms_contrast` e `sharpness` risultino soggette a drift, seppur con distanza di Wasserstein più bassa rispetto ad altri tipi di sporcatura. Ciò è riportato in figura 5.19. Questo è dato probabilmente dal fatto che le feature estratte per questo tipo di sporcature presentino un drift minimo o nullo rispetto alle feature di riferimento.
- **pixelate:** A differenza di quanto rilevato per le feature visive `rms_contrast` e `sharpness`, riportato in figura 5.18, le feature estratte dal modello presentano drift solo nel primo livello di severity. Valgono quindi le considerazioni fatte in precedenza.

Noise

corruption	severity	is_drift	p_val	distance
gaussian_noise	1	False	0.5000	0.0000
gaussian_noise	2	True	0.0000	0.0005
gaussian_noise	3	True	0.0000	0.0025
gaussian_noise	4	True	0.0000	0.0110
gaussian_noise	5	True	0.0000	0.0682
impulse_noise	1	True	0.0000	0.0004
impulse_noise	2	True	0.0000	0.0014
impulse_noise	3	True	0.0000	0.0040
impulse_noise	4	True	0.0000	0.0215
impulse_noise	5	True	0.0000	0.0840
shot_noise	1	False	0.2000	0.0000
shot_noise	2	True	0.0000	0.0005
shot_noise	3	True	0.0000	0.0028
shot_noise	4	True	0.0000	0.0155
shot_noise	5	True	0.0000	0.0433

Tabella 5.8: Risultati del test di MMD tra le feature estratte dal layer `avgpool` del modello ResNet-50 per ImageNet-1k e ImageNet-C, corruzione di tipo noise.

Per quanto riguarda la classe di sporcature noise, abbiamo ottenuto i risultati presentati in tabella 5.8. Possiamo effettuare le seguenti osservazioni:

- **gaussian_noise:** Per questo tipo di sporcatura, le feature visive `rms_contrast`, `sharpness` presentano sempre un drift, le medie dei canali colore presentano un drift dal quarto livello di severità in poi e `brightness` presenta un drift dal terzo livello di severità in poi. Questi risultati sono riportati in figura 5.6. A differenza di quanto rilevato nelle feature visive, le feature estratte dal modello presentano un drift solamente dal secondo livello di severità. Come quindi già detto in precedenza, le feature estratte dal modello per il primo livello di severità sono uguali a quelle estratte per ImageNet-1k, infatti la distanza MMD rilevata è 0.
- **impulse_noise:** Per questo tipo di sporcatura, le feature visive `rms_contrast` e `sharpness` presentano sempre un drift, le medie dei canali colore presentano un drift dal quarto livello di severità e `brightness` dal secondo livello di severità. Ciò è riportato nella figura 5.8. Come rilevato in precedenza, vi è quindi una discrepanza tra il drift rilevato nelle feature visive e quello rilevato nelle feature estratte dal modello: infatti, quest'ultime presentano sempre un drift rispetto alle feature di riferimento.
- **shot_noise:** Per questo tipo di sporcatura, le feature visive `rms_contrast` e `sharpness` presentano sempre un drift, `brightness` presenta un drift dal terzo livello di severità in poi. A differenza degli altri tipi di sporcatura in questa classe, l'unica media del canale colore che presenta un drift è quella del canale blu all'ultimo livello di severità. Anche in questo caso, il drift rilevato nelle feature estratte dal modello diverge rispetto a quanto rilevato nelle feature visive: infatti, viene rilevato un drift solamente dal secondo livello di severità.

Weather

corruption	severity	is_drift	p_val	distance
brightness	1	False	0.3500	0.0000
brightness	2	False	0.1000	0.0001
brightness	3	False	0.1000	0.0000
brightness	4	False	0.2500	0.0001
brightness	5	True	0.0000	0.0003
fog	1	False	0.3500	0.0000
fog	2	False	0.1500	0.0001
fog	3	False	0.2500	0.0001
fog	4	False	0.1000	0.0002
fog	5	True	0.0000	0.0021
frost	1	False	0.1500	0.0000
frost	2	True	0.0000	0.0004
frost	3	True	0.0000	0.0016
frost	4	True	0.0000	0.0020
frost	5	True	0.0000	0.0043
snow	1	True	0.0000	0.0003
snow	2	True	0.0000	0.0010
snow	3	True	0.0000	0.0032
snow	4	True	0.0000	0.0050
snow	5	True	0.0000	0.0133

Tabella 5.9: Risultati del test di MMD tra le feature estratte dal layer `avgpool` del modello ResNet-50 per ImageNet-1k e ImageNet-C, corruzione di tipo weather.

Per quanto riguarda la classe di sporcature weather, abbiamo ottenuto i risultati presentati in tabella 5.9. Possiamo effettuare le seguenti osservazioni:

- **brightness:** Per questo tipo di sporcatura, si presenta un drift in tutte le feature visive e a tutti i livelli di severity tranne per le medie dei canali colore, le quali non presentano alcun tipo di drift. Ciò è riportato in figura 5.16. Come si può osservare dalla tabella 5.9, le feature estratte dal modello presentano un drift rispetto a quelle estratte da ImageNet-1k solamente al quinto livello di severity. Valgono quindi le considerazioni effettuate in precedenza sulle feature estratte dal modello.
- **fog:** Per questo tipo di sporcatura, tutte le feature visive presentano un drift a tutti i livelli di severity. La figura 5.14 presenta il drift misurato su queste feature. Ciò non si riflette nelle feature estratte dal modello, nelle quali viene rilevato un drift dal test MMD solamente al quinto livello di severity.
- **frost:** Per questo tipo di sporcatura, tutte le feature visive presentano un drift a tutti i livelli di severity. La figura 5.13 presenta il drift misurato su queste feature. Ciò si riflette solamente parzialmente sulle feature estratte dal modello: infatti, viene rilevato un drift in esse solamente dal secondo livello di severity.
- **snow:** Per questo tipo di sporcatura, tutte le feature visive presentano un drift a tutti i livelli di severity tranne `rms_contrast`, la quale presenta un drift solamente a partire dal secondo livello di severity. La figura 5.15 presenta il drift misurato su queste feature. Ciò si riflette quindi solamente parzialmente sulle feature estratte dal modello: infatti, viene rilevato un drift in esse a tutti i livelli di severity.

Analisi del confronto tra tipi di feature

Confrontando i risultati ottenuti per la rilevazione del drift sulle feature visive e sulle feature estratte dal modello, si può evincere che non vi è una correlazione diretta tra la presenza di drift nelle due. Ciò può essere dato dal fatto che la rete convolutiva computa e utilizza feature map le quali non sono direttamente confrontabili con le feature visive direttamente estratte dall'immagine, nonostante esse possano venir utilizzate per calcolare le feature map. Il drift delle feature del modello non sembra quindi essere influenzato dal drift nelle feature visive.

5.3 Analisi delle performance del modello

In questa sezione vengono analizzate le performance del modello ResNet-50 su ImageNet-R e ImageNet-C. Per ImageNet-R, confronteremo le metriche ottenute dalla classificazione delle immagini del dataset con le metriche ottenute dalla classificazione delle immagini di ImageNet-1k. Per ImageNet-C, invece, analizzeremo l'accuracy del modello al variare della severità della corruzione, calcolando la degradazione delle performance rispetto alla severity, calcolando la robustezza del modello e misurando quanto linearmente la severità della corruzione influisca sulla degradazione delle performance del modello.

5.3.1 ImageNet-1k vs ImageNet-R

Per ottenere dei valori statisticamente significativi, abbiamo ripetuto la classificazione per cinque volte, in modo da ottenere più misurazioni delle metriche e calcolare gli intervalli di confidenza. I risultati ottenuti sono presentati in tabella 5.10. In particolare, la colonna Sig. indica la significatività statistica, secondo il test t Welch, del delta tra le due misure ottenute. Come si può notare dalla tabella, tutte

Metric	ImageNet-1k (mean)	ImageNet-R (mean)	Delta (mean $\pm$ CI)	Sig.
Cohen's kappa	0.8426	0.3021	0.5404 [0.5390, 0.5419]	YES
Accuracy	0.8433	0.3034	0.5398 [0.5383, 0.5413]	YES
Balanced accuracy	0.8433	0.3034	0.5398 [0.5383, 0.5413]	YES
Recall weighted	0.8433	0.3034	0.5398 [0.5383, 0.5413]	YES
Precision micro	0.8433	0.3034	0.5398 [0.5383, 0.5413]	YES
Recall micro	0.8433	0.3034	0.5398 [0.5383, 0.5413]	YES
F1 micro	0.8433	0.3034	0.5398 [0.5383, 0.5413]	YES
MCC	0.8427	0.3035	0.5392 [0.5377, 0.5407]	YES
Top-5 accuracy	0.9635	0.4572	0.5063 [0.5054, 0.5072]	YES
F1 weighted	0.8962	0.4178	0.4784 [0.4766, 0.4803]	YES
Recall macro	0.2440	0.0725	0.1715 [0.1694, 0.1735]	YES
F1 macro	0.2593	0.0998	0.1595 [0.1574, 0.1616]	YES
Precision weighted	0.9689	0.8184	0.1504 [0.1466, 0.1543]	YES
Precision macro	0.2804	0.1956	0.0848 [0.0820, 0.0875]	YES

Tabella 5.10: Confronto tra le metriche di performance del modello

le metriche registrare calano significativamente quando il modello si trova a classificare i dati affetti da drift. Infatti, ImageNet-R presenta un drift sia sulle feature visive (cinque feature su sei presentano drift) sia sulle feature del modello. In particolare, l'accuracy, su cui ci siamo concentrati durante l'analisi di ImageNet-C, cala drasticamente, registrando un decremento del 53.9%.

5.3.2 ImageNet-1k vs ImageNet-C

Per il confronto con i risultati ottenuti su ImageNet-C, abbiamo misurato l'accuracy al variare della severità per ogni classe di corruzione e l'abbiamo confrontata con l'accuracy ottenuta da ImageNet-1k. Abbiamo inoltre misurato la degradazione dell'accuracy al variare della severità della corruzione, calcolato l'AUC della robustezza al variare della severità e calcolato il coefficiente R^2 della degradazione.

Blur

Corruption Type	AUC	Slope	R^2
Motion blur	0.561179	0.171572	0.985192
Zoom blur	0.545493	0.097476	0.994757
Defocus blur	0.534979	0.155609	0.986706
Glass blur	0.369184	0.176186	0.904344

Tabella 5.11: Metriche di performance per la classificazione con corruzione di tipo blur

Come si può notare dal grafico in figura 5.22, la degradazione dell'accuracy in generale cresce al variare della severity. Ciò è coerente con l'andamento dell'accuracy riportata in figura 5.21. In particolare, il

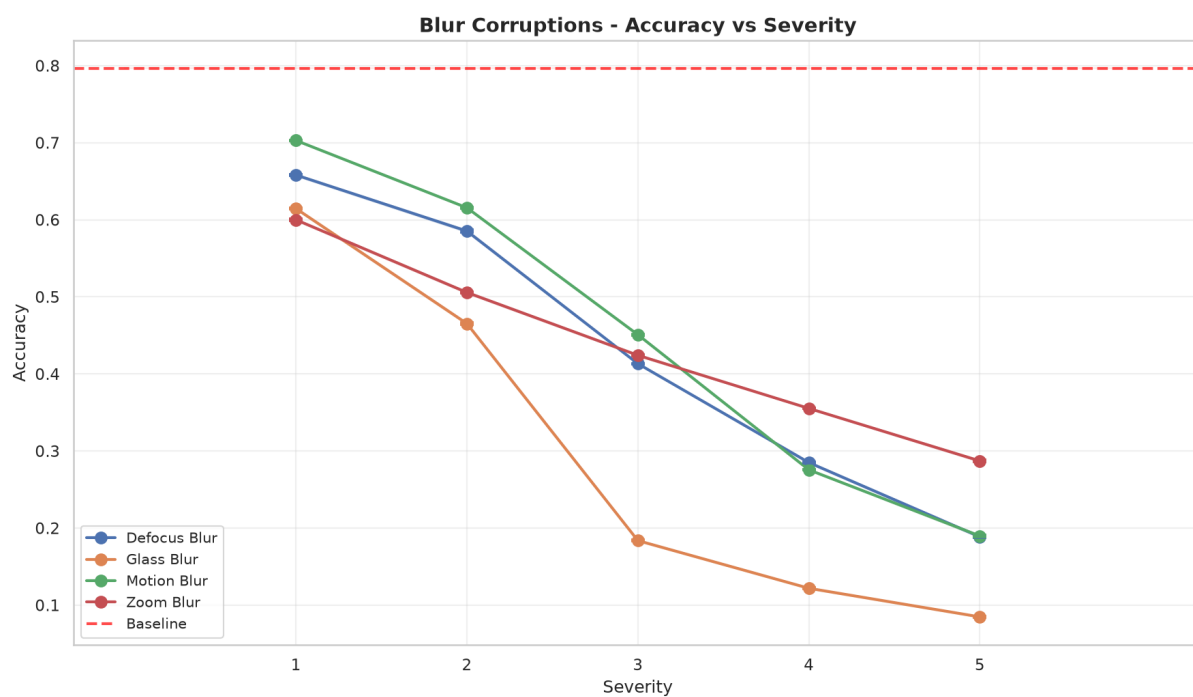


Figura 5.21: Valore dell'accuracy al variare della severità per corruzioni di tipo blur

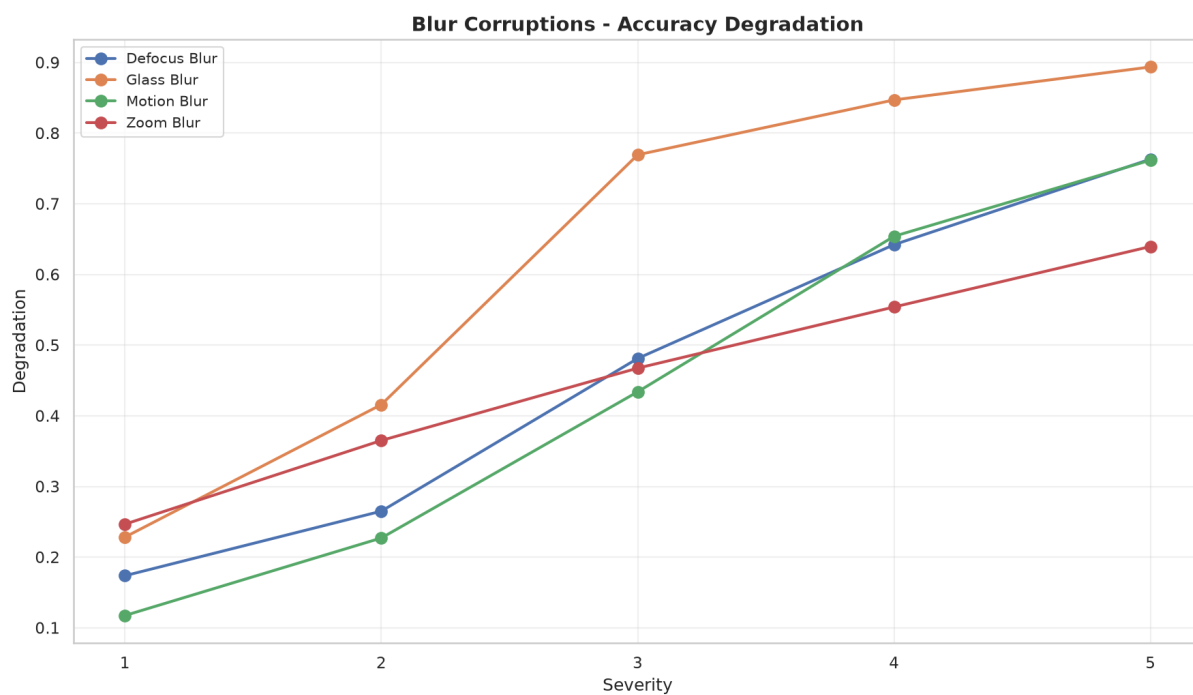


Figura 5.22: Degradazione dell'accuracy al variare della severità per corruzioni di tipo blur

coefficiente R^2 calcolato per `glass_blur` indica che la degradazione cresce meno linearmente rispetto agli altri tipi di blur. Infatti, si registra un aumento repentino della degradazione dalla severity 3. Inoltre, al livello di severity 5, il valore di accuracy per `glass_blur` raggiunge un valore inferiore a 0.1, infatti presenta una AUC della robustezza molto bassa. D'altro canto, il coefficiente R^2 calcolato per `zoom_blur` indica che la degradazione delle performance cresce linearmente con il livello di severity, indicando una correlazione tra degradazione e severità per questo tipo di sporatura.

Digital

Corruption Type	AUC	Slope	R^2
JPEG compression	0.770172	0.056896	0.925757
Contrast	0.731614	0.131767	0.846042
Pixelate	0.708785	0.114113	0.941701
Elastic transform	0.605682	0.153883	0.873604

Tabella 5.12: Metriche di performance per la classificazione con corruzione di tipo digital

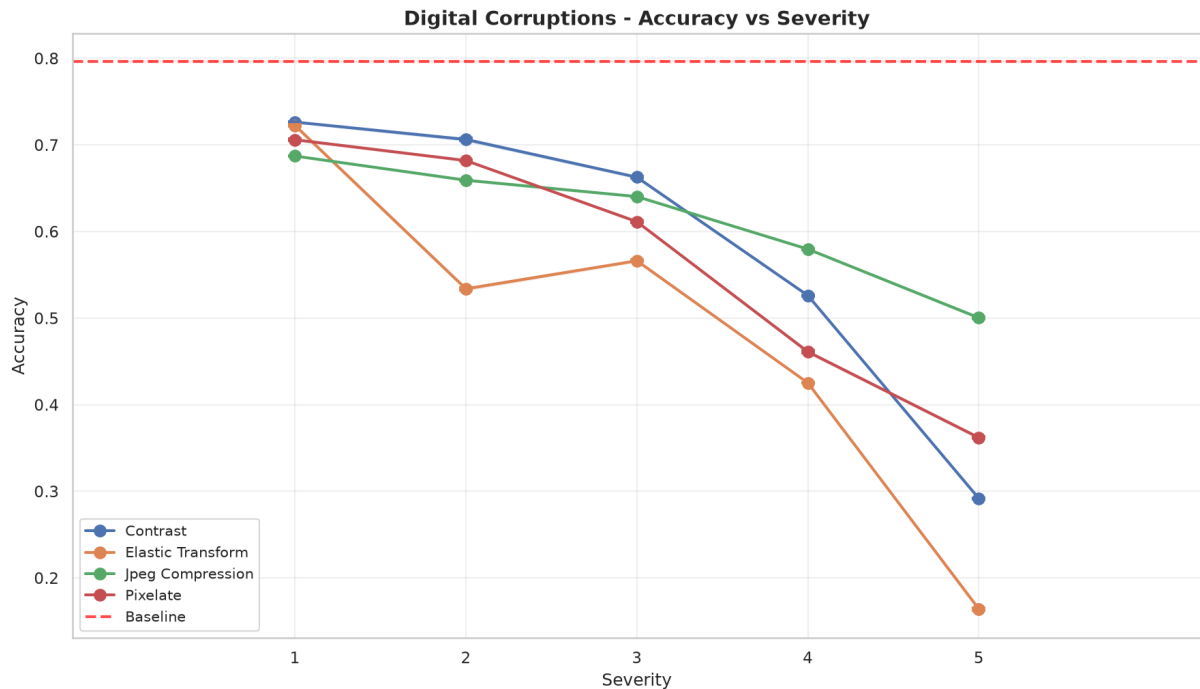


Figura 5.23: Valore dell'accuracy al variare della severità per corruzioni di tipo digital

Come si può notare dal grafico in figura 5.23, l'accuracy decresce con il livello di severità della corruzione eccezion fatta per `elastic_transform`, la quale presenta un breve incremento dell'accuracy passando dal livello due al livello tre di severità. I risultati ottenuti si riflettono nei valori di robustezza del modello, presentati in figura 5.24. In particolare, la degradazione delle performance non segue affatto un andamento lineare per `contrast` e `elastic_transform`, mentre per `JPEG_compression` e `pixelate` sono più vicini ad un andamento lineare.

Noise

Come si può notare dalla tabella 5.13 e dalle figure 5.25 e 5.26, il calo di performance e l'aumento della degradazione seguono un andamento molto vicino ad un andamento lineare. Le performance del modello per questa classe di sporatura calano molto al variare della severity, arrivando addirittura a un valore di quasi 0.1 di accuratezza a livello di severità 5. I valori AUC riflettono questo calo, arrivando nel caso di `impulse_noise` ad un valore inferiore a 0.5.

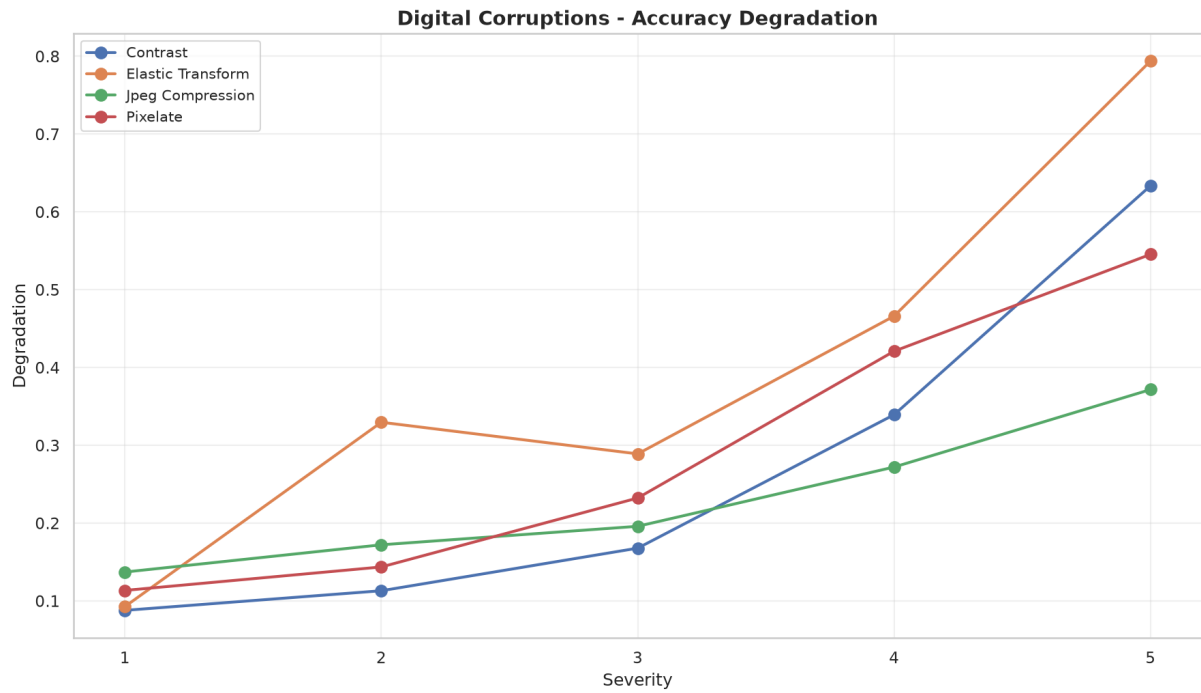


Figura 5.24: Degradazione dell'accuracy al variare della severità per corruzioni di tipo digital

Corruption Type	AUC	Slope	R ²
Gaussian noise	0.535206	0.184893	0.979393
Shot noise	0.512519	0.182769	0.983590
Impulse noise	0.482528	0.163862	0.970980

Tabella 5.13: Metriche di performance per la classificazione con corruzione di tipo noise

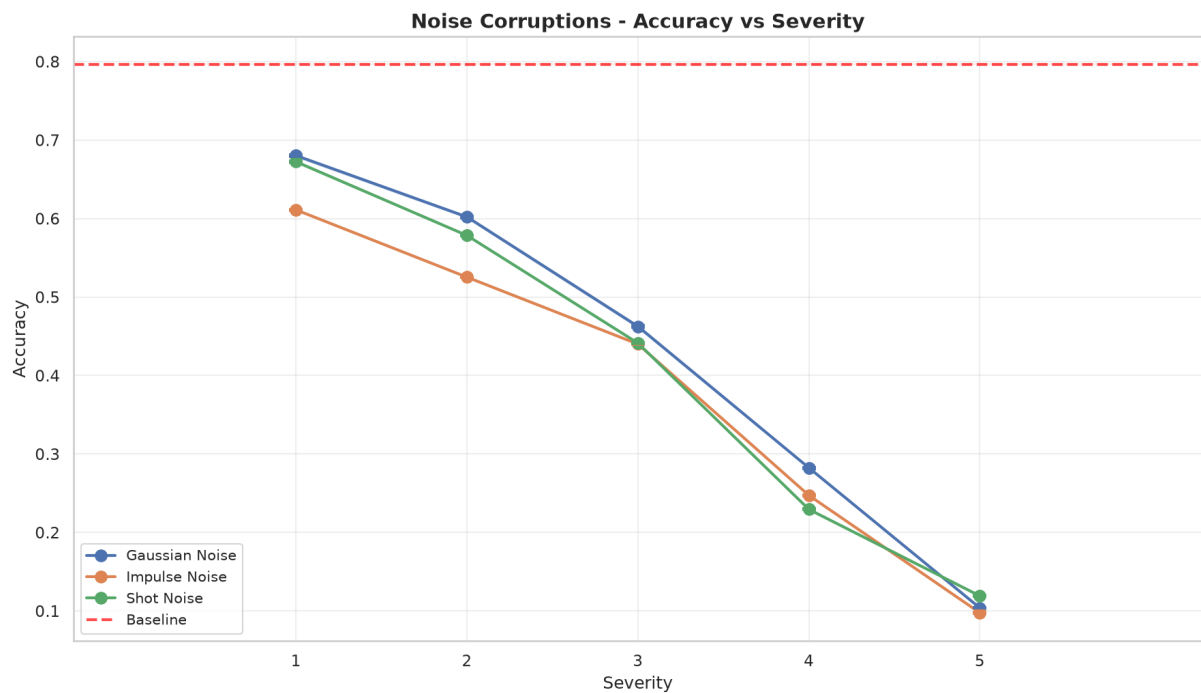


Figura 5.25: Valore dell'accuracy al variare della severità per corruzioni di tipo digital

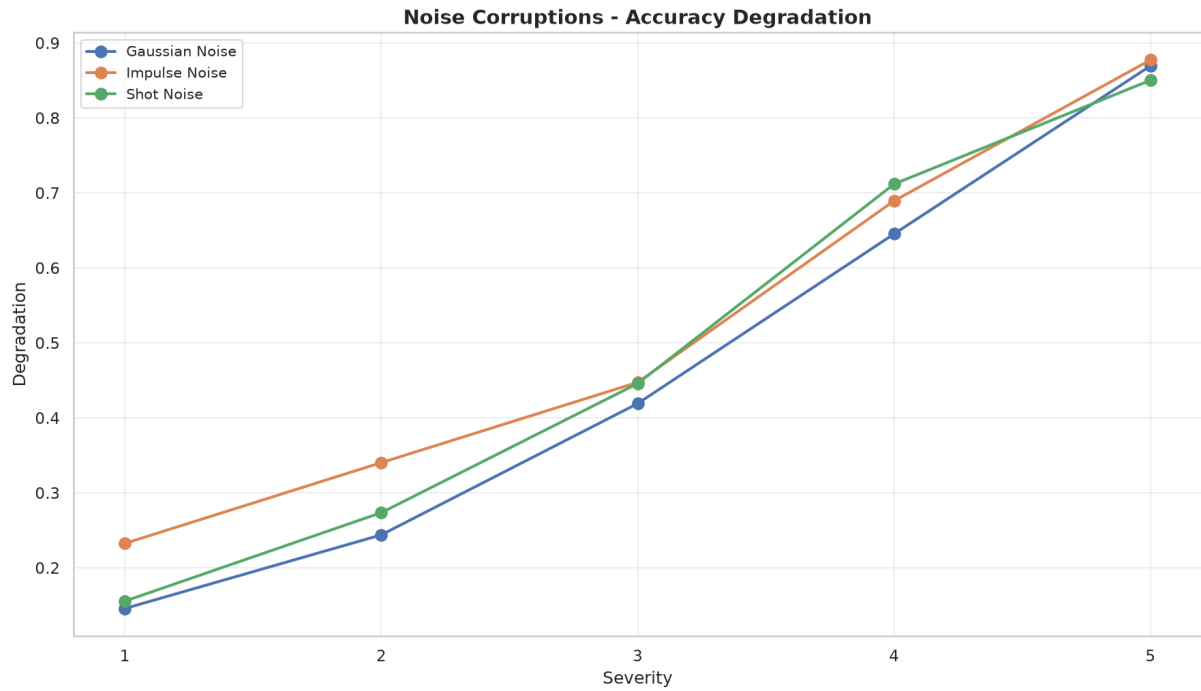


Figura 5.26: Degradazione dell'accuracy al variare della severità per corruzioni di tipo noise

Weather

Corruption Type	AUC	Slope	R ²
Brightness	0.899717	0.034920	0.932987
Fog	0.736412	0.097285	0.892568
Frost	0.574758	0.102072	0.924922
Snow	0.503722	0.097529	0.865135

Tabella 5.14: Metriche di performance per la classificazione con corruzione di tipo weather.

Come si può notare dai valori in tabella 5.14 e dalle figure 5.27 e 5.28, l'accuratezza registrata dal modello decresce per ogni tipo sporcaturo e per ogni livello di severity. Tuttavia, il decremento dell'accuracy è decisamente meno marcato per brightness, su cui il modello riesce a mantenere performance relativamente elevate rispetto agli altri tipi di sporcaturo, mantenendo anche un AUC della robustezza molto elevata. Inoltre, la degradazione delle performance per brightness segue un andamento quasi lineare ma con un'inclinazione bassa. Inoltre, nonostante il calo dell'accuracy, fog mantiene comunque un'AUC della robustezza moderatamente elevata, indicando quindi resistenza maggiore a questo tipo di sporcaturo.

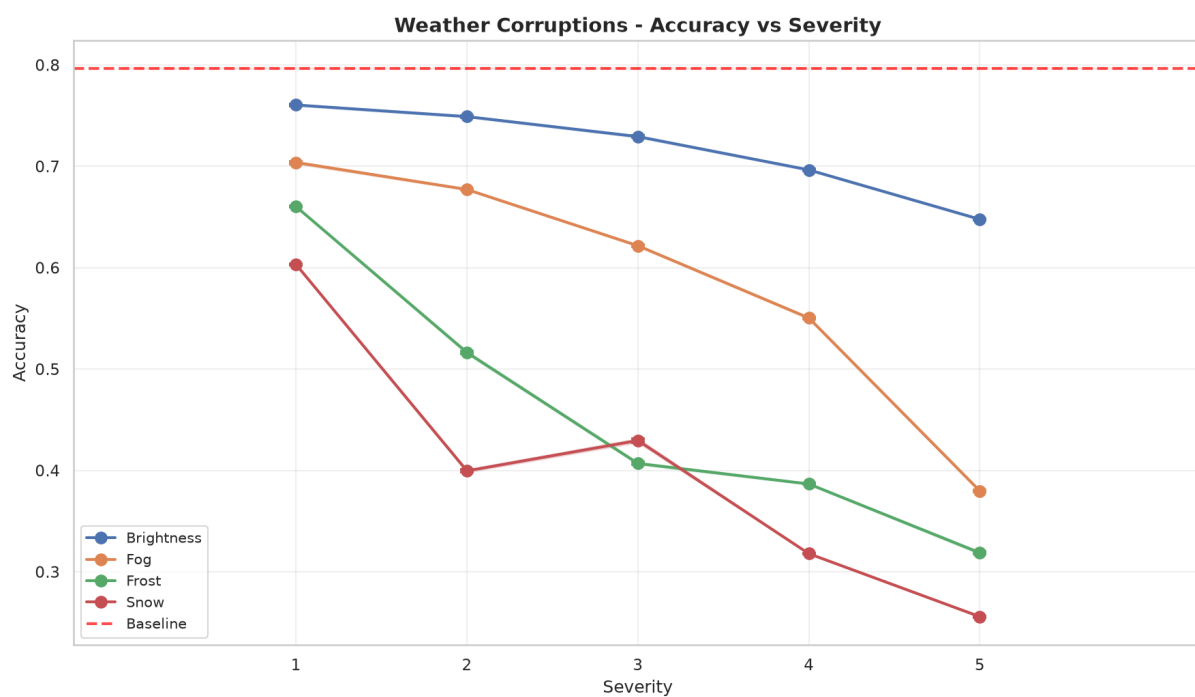


Figura 5.27: Valore dell'accuracy al variare della severità per corruzioni di tipo weather

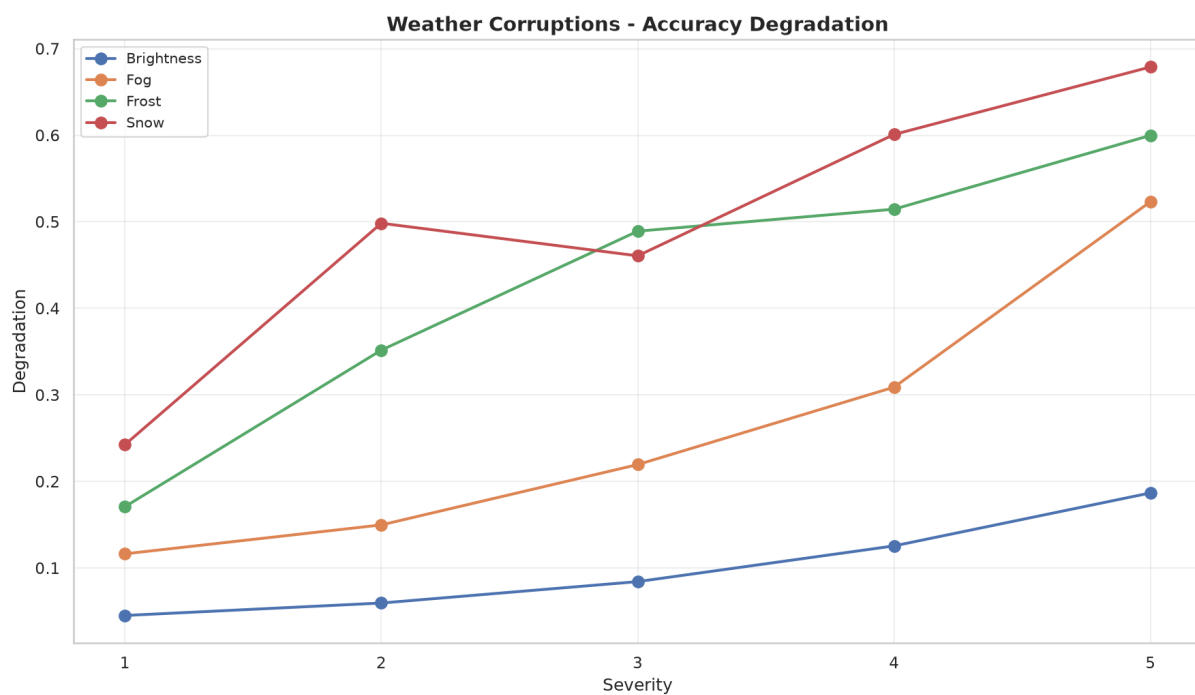


Figura 5.28: Degradazione dell'accuracy al variare della severità per corruzioni di tipo weather

Bibliografia

- [1] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla e F. Herrera, «A unifying view on dataset shift in classification,» *Pattern Recognition*, vol. 45, n. 1, pp. 521–530, 1 gen. 2012, ISSN: 0031-3203. DOI: 10.1016/j.patcog.2011.06.019 visitato il giorno 22 ago. 2026. indirizzo: <https://www.sciencedirect.com/science/article/pii/S0031320311002901>
- [2] L. Tamang, M. R. Bouadjenek, R. Dazeley e S. Aryal, «Handling Out-of-Distribution Data: A Survey,» *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, n. 10, pp. 5948–5966, ott. 2025, ISSN: 1558-2191. DOI: 10.1109/TKDE.2025.3592614 visitato il giorno 22 ago. 2026. indirizzo: <https://ieeexplore.ieee.org/document/11098614/>
- [3] K. He, X. Zhang, S. Ren e J. Sun, «Deep Residual Learning for Image Recognition,» in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, ISSN: 1063-6919, giu. 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90 visitato il giorno 22 ago. 2026. indirizzo: <https://ieeexplore.ieee.org/abstract/document/7780459>
- [4] D. Hendrycks e T. G. Dietterich, *Benchmarking Neural Network Robustness to Common Corruptions and Surface Variations*, 27 apr. 2019. DOI: 10.48550/arXiv.1807.01697 arXiv: 1807.01697[cs.LG]. visitato il giorno 24 ago. 2026. indirizzo: <http://arxiv.org/abs/1807.01697>
- [5] D. Hendrycks et al., «The Many Faces of Robustness: A Critical Analysis of Out-of-Distribution Generalization,» in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, ISSN: 2380-7504, ott. 2021, pp. 8320–8329. DOI: 10.1109/ICCV48922.2021.00823 visitato il giorno 22 ago. 2026. indirizzo: <https://ieeexplore.ieee.org/document/9710159/>
- [6] Y. LeCun et al., «Handwritten Digit Recognition with a Back-Propagation Network,» in *Advances in Neural Information Processing Systems*, vol. 2, Morgan-Kaufmann, 1989. visitato il giorno 4 set. 2026. indirizzo: https://proceedings.neurips.cc/paper_files/paper/1989/hash/53c3bce66e43be4f209556518c2fcb54-Abstract.html
- [7] K. O’Shea e R. Nash. «An introduction to convolutional neural networks,» arXiv.org, visitato il giorno 4 set. 2026. indirizzo: <https://arxiv.org/abs/1511.08458v2>
- [8] J. Gu et al., «Recent advances in convolutional neural networks,» *Pattern Recognition*, vol. 77, pp. 354–377, 1 mag. 2018, ISSN: 0031-3203. DOI: 10.1016/j.patcog.2017.10.013 visitato il giorno 5 set. 2026. indirizzo: <https://www.sciencedirect.com/science/article/pii/S0031320317304120>
- [9] Y. LeCun, Y. Bengio e G. Hinton, «Deep learning,» *Nature*, vol. 521, n. 7553, pp. 436–444, mag. 2015, ISSN: 1476-4687. DOI: 10.1038/nature14539 visitato il giorno 5 set. 2026. indirizzo: <https://www.nature.com/articles/nature14539>